

Enterprise AI Is a Platform Problem: GKS-5, A Reference Architecture for Governed Knowledge Systems

Roopam W. Sure

Independent Researcher

Ottawa, Ontario, Canada

Email: roopam.walia.sure@gmail.com

ORCID: 0009-0008-3720-2497

Author Note: This manuscript was prepared independently. The views expressed are solely those of the author and do not represent any employer, client, or affiliated organization.

Abstract

Enterprise adoption of generative AI has outpaced the knowledge infrastructure needed to operate AI systems reliably. A team can build an impressive assistant over a curated folder, a small policy corpus, or a narrow set of service tickets. The harder work begins when the same assistant is expected to operate across document repositories, SaaS platforms, collaboration tools, business applications, and data estates with uneven governance. In that setting, many failures that look like model failures are actually failures in the knowledge path around the model.

This article introduces GKS-5, a five-layer reference architecture for Governed Knowledge Systems in enterprise AI. GKS-5 separates five responsibilities: data ingestion, storage and semantic indexing, retrieval, model orchestration, and application integration. It then pairs those layers with six cross-layer controls: metadata, authorization, freshness, observability, evaluation, and versioning. The purpose is to reduce knowledge integrity drift, defined here as the gradual loss of alignment between source systems, metadata, permissions, indexes, retrieval behavior, orchestration logic, and the outputs users see.

The contribution is a practical architecture and evaluation framework for enterprise AI knowledge systems. It includes a failure-mode taxonomy, a readiness scoring model, an evaluation protocol, and a scenario-based validation using the common case of an internal policy assistant moving from pilot to production. The core claim is deliberately simple: enterprise assistants are rarely durable products by themselves. They are interaction surfaces over deeper knowledge platforms. Better models help, but they do not remove the need for governed ingestion, authorization-aware retrieval, source authority, traceable context assembly, and systematic evaluation.

Keywords: *Enterprise AI; Governed Knowledge Systems; GKS-5; Retrieval-Augmented Generation; Knowledge Architecture; Semantic Retrieval; AI Governance; Agentic Systems; Knowledge Integrity Drift; AI Platform Architecture*

1. Introduction

Enterprise AI has moved beyond the question that dominated the first wave of generative AI adoption. The question is no longer whether large language models can produce plausible answers over enterprise content. They can. The harder question is whether those answers can be delivered with reliability, provenance, access control, freshness, auditability, and operational discipline at enterprise scale.

Across industries, organizations are deploying internal copilots, policy assistants, knowledge search tools, document intelligence systems, and domain-specific conversational interfaces. These efforts usually begin for practical reasons. Business teams do not want to wait for multi-year data modernization programs before exposing institutional knowledge through natural language. They want useful AI capabilities now, even when the underlying content estate is fragmented, duplicated, unevenly governed, or poorly described.

That urgency creates a structural mismatch. Enterprise data platforms mature through long-running programs focused on integration, storage, lineage, cataloging, quality, security, and governance. Generative AI initiatives usually operate on shorter timelines. Teams are asked to deliver assistants, copilots, and workflow agents in weeks or months.

As a result, AI systems are often built outside, beside, or ahead of the governed data ecosystems they eventually need to rely on.

In pilots, this mismatch is easy to miss. A small, curated corpus can make a weak architecture look stronger than it is. A retrieval-augmented generation pipeline over selected documents may produce convincing answers [1]. A conversational interface over a departmental repository may create visible momentum. But once the system moves beyond the pilot boundary, hidden assumptions become operational problems. Freshness varies by source. Permissions change. Documents are duplicated or superseded. Metadata is missing or inconsistent. Retrieval quality declines as the corpus grows. Observability is too thin to explain why a particular answer was produced. The application becomes dependent on whichever vector store, orchestration framework, or model endpoint was selected during the first experiment.

At that stage, the failure is often misdiagnosed. Users may describe the system as hallucinating, and teams may respond by changing prompts, switching models, or adding more context. Sometimes those changes help. Often, they do not address the underlying cause. The system may have retrieved the wrong document, ignored a permission boundary, ranked a stale source above an authoritative one, or lost the metadata needed to interpret the answer. What appears to be a model problem is frequently a platform problem.

This article introduces GKS-5, a five-layer reference model for Governed Knowledge Systems in enterprise AI. A Governed Knowledge System is an enterprise AI architecture designed to operate over heterogeneous, permissioned, and continuously changing organizational knowledge. GKS-5 separates five responsibilities: data ingestion, storage and semantic indexing, retrieval, model orchestration, and application integration. These layers are paired with cross-layer controls for metadata, authorization, freshness, observability, evaluation, and versioning.

GKS-5 is designed around knowledge integrity drift: the gradual loss of alignment between source systems, metadata, permissions, indexes, retrieval behavior, orchestration logic, and the outputs users see. A system can still appear operational while this drift is occurring. Documents may be indexed, answers may be generated, and users may receive fluent responses. Yet the knowledge path may no longer be trustworthy. Sources may be stale. Permissions may be out of date. Ranking may favor informal content over authoritative sources. Prompts may have changed without evaluation. Citations may no longer support the claims they accompany.

The stakes are higher in regulated, public-sector, and operationally sensitive environments, where AI risk management, security controls, and access-control expectations are more explicit [19], [20], [21]. Financial institutions, healthcare organizations, government agencies, and critical infrastructure operators cannot treat governance as a presentation-layer concern added after a successful demo. They need systems that preserve provenance, enforce entitlements, support auditability, handle sensitive information correctly, and degrade safely when content is stale, incomplete, or outside policy. In these settings, early shortcuts are often repaid later as rework, migration cost, compliance exposure, and loss of user trust.

This distinction matters because durable systems are rarely the ones with the most impressive early demonstrations. They are the ones with clear source ownership, reliable metadata, authorization boundaries that survive retrieval and generation, retrieval strategies matched to enterprise queries, evaluation practices that detect regressions, and interfaces that allow individual components to evolve. In enterprise AI, poor metadata can be more damaging than a weaker model because it affects what the system can find, what it is allowed to retrieve, how it ranks evidence, and whether users can trust the result.

GKS-5 should therefore be understood as a structural model, not a complete maturity claim on its own. A system may implement all five layers and still fail if it loses provenance during ingestion, retrieves restricted content, ranks stale sources above authoritative ones, lacks evaluation coverage, or cannot trace an answer back to its supporting evidence. For that reason, the five layers must be paired with controls that preserve knowledge integrity across the full path from source systems to generated outputs or executed actions.

1.1 Contributions

The article makes five contributions:

1. It defines Governed Knowledge Systems as an architectural class for enterprise AI systems that operate over heterogeneous, permissioned, and continuously changing organizational knowledge.

2. It introduces GKS-5, a five-layer reference model that separates data ingestion, storage and semantic indexing, retrieval, model orchestration, and application integration.
3. It identifies six cross-layer controls required to make GKS-5 ready for production: metadata, authorization, freshness, observability, evaluation, and versioning.
4. It proposes a failure-mode taxonomy showing how user-visible failures such as hallucinations, stale answers, citation errors, and unsafe actions often originate in specific platform layers or controls.
5. It introduces a Governed Knowledge System readiness model and evaluation protocol that organizations can use to assess whether an AI knowledge system is pilot-ready, ready for production, or suitable for controlled agentic workflows.

1.2 Practitioner motivation and scope

The motivation for this architecture is not a single product stack or a claim that every enterprise should standardize on one implementation. It comes from a repeated delivery pattern: the first AI assistant succeeds because the corpus is small, the users are close to the project team, and the governance assumptions are implicit. The second and third use cases then expose the real platform problem. A new repository has different permissions. A policy has an older copy in another location. A support workflow needs exact identifiers rather than semantic similarity. A business user expects a citation, not only a fluent paragraph.

That pattern is why the paper uses architecture as the unit of analysis. The practical question is not whether a model can answer a question over a set of documents. It can. The practical question is whether the enterprise can explain which sources were eligible, which passages were retrieved, whether the evidence was current, whether the user was entitled to it, which prompt or workflow version was used, and what should happen when the answer is not well supported.

2. Background and Related Work

2.1 Evolution of Enterprise Data Platforms

Enterprise data architecture has evolved around the dominant workloads organizations needed to support. Data warehouses were designed for structured analytics, business intelligence, reporting, and repeatable decision support. They worked well where data could be modeled into curated schemas, transformations were deterministic, governance was centralized, and users asked known questions over well-understood metrics.

Data lakes changed the operating model. They allowed enterprises to retain raw, semi-structured, and unstructured data at larger scale without forcing every asset into a relational schema upfront. This improved flexibility and reduced storage friction, but it also created a familiar management problem: data became easier to collect than to trust. Metadata was incomplete, ownership was unclear, quality varied by source, and duplicated or poorly described assets weakened confidence in downstream consumption.

The response was to reintroduce structure around the lake. Catalogs, lineage systems, data quality monitoring, governance controls, and lakehouse patterns emerged because raw storage alone was not enough [12], [13]. More recent architectural ideas extended that direction. Data fabric emphasized integration, metadata, interoperability, and governance across distributed environments [11]. Data mesh emphasized domain ownership, accountability, and decentralized responsibility for data products [9], [10], [11].

These developments strengthened enterprise analytics and governance. They remain important foundations for AI systems. But they were not designed primarily for retrieval-time reasoning over heterogeneous enterprise knowledge. Traditional data platforms are usually optimized for structured queries, known transformations, stable metrics, and deterministic processing. Enterprise AI knowledge systems behave differently, especially when retrieval, semantic representation, and knowledge context become part of the runtime path [1], [14]. They retrieve from structured, semi-structured, and unstructured sources; assemble context dynamically at query time; combine semantic similarity with lexical search and metadata filters; and rely on model reasoning to produce or support an answer.

The gap becomes visible when an enterprise assistant moves beyond a narrow pilot corpus. A conventional data platform may know where a document lives, who owns a dataset, and when a table was refreshed. A governed AI knowledge system must answer different questions: Which passage is authoritative for this user? Is this document current or superseded? Does the user have permission to retrieve this content before it reaches the model? Should the

system prefer an exact policy clause, a semantically similar explanation, or a more recent operational record? How should conflicting sources be ranked, cited, or excluded?

These questions require capabilities that sit between traditional data management and model inference. They require ingestion pipelines that preserve provenance and permissions, indexing strategies that support both semantic and exact retrieval, chunk-level metadata that survives downstream use, and observability that explains how retrieved evidence shaped a generated response. The evolution from warehouses to lakes to lakehouses, fabrics, and meshes provides important infrastructure. It does not, on its own, provide the full architecture required for governed enterprise AI knowledge systems.

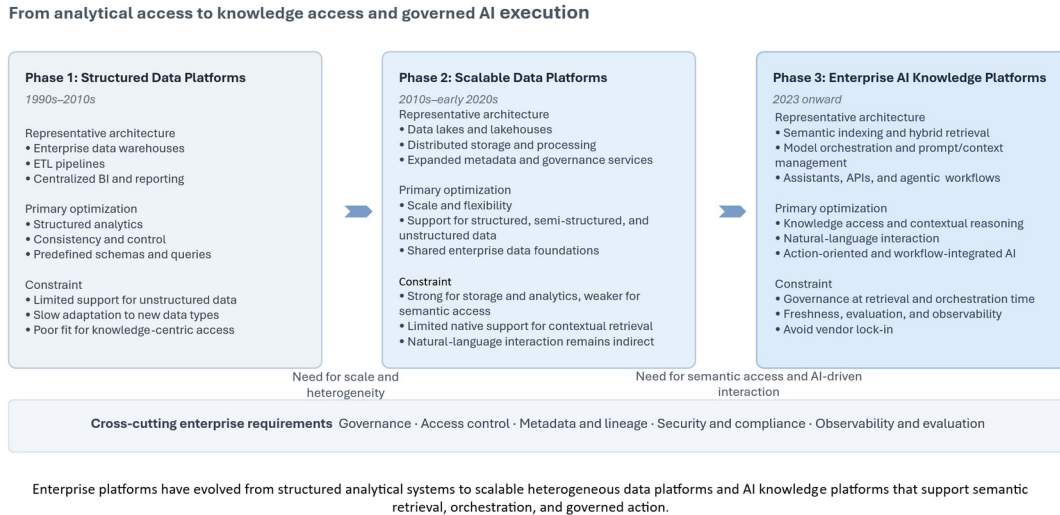


Figure 1. Evolution of enterprise data and AI platform architectures. Enterprise platforms have evolved from structured analytical systems to scalable heterogeneous data platforms and AI knowledge platforms that support semantic retrieval, orchestration, and governed action.

2.2 Retrieval-Augmented Generation and Enterprise Knowledge Access

Retrieval-augmented generation, commonly known as RAG, has become the dominant pattern for grounding language models in enterprise content [1]. The appeal is practical. Instead of retraining a model on proprietary, sensitive, or frequently changing information, an organization retrieves relevant context from external systems and supplies that context to the model at inference time. The source of truth remains outside the model's parameters, while users interact with enterprise knowledge through natural language.

For many enterprise use cases, this pattern is necessary. Policies, manuals, tickets, reports, wikis, procedures, knowledge bases, and operational records change too often, or carry too much governance risk, to be treated simply as training data. RAG gives organizations a path to build useful assistants before completing broader data modernization. It can ground responses in current sources, support citations, and reduce dependence on model memory.

The problem is that RAG is often implemented as if retrieval were simple. A common first architecture includes document ingestion, chunking, embeddings, a vector database, a prompt template, and a model endpoint, building on dense retrieval and large-scale similarity-search techniques [2], [6]. This can work in a constrained pilot, especially when the corpus is small and manually curated. It becomes less reliable when the system encounters the actual shape of enterprise knowledge: duplicate documents, inconsistent metadata, changing permissions, conflicting source authority, domain-specific abbreviations, exact identifiers, temporal constraints, and questions that combine business intent with operational detail.

In these environments, the generation step is not always the first component to fail. More often, the failure starts upstream. The system retrieves a plausible but outdated document. It misses an exact policy clause because semantic similarity is not enough. It ranks an informal page above an approved source. It includes content that should have been excluded by entitlement rules. It assembles context that is individually relevant but collectively misleading. The final answer may look like a hallucination, but the underlying failure is frequently in retrieval, metadata, freshness, authorization, or context assembly.

Enterprise queries are rarely pure semantic questions. A user may ask about a product code, a regional exception, a contractual clause, a control procedure, a ticket number, or a recent operational incident. These questions often require hybrid retrieval: semantic search, lexical search, metadata-aware filtering, ranking, deduplication, source authority, and context assembly working together [2], [3], [4], [5]. Vector search is valuable, but it is not sufficient as the retrieval layer for enterprise AI.

The distinction is practical. RAG is a grounding pattern, not, on its own, an enterprise architecture. It does not, on its own, define ingestion contracts, metadata standards, authorization propagation, index lifecycle management, source authority, evaluation practices, observability, or rollback behavior when retrieval quality degrades. A working RAG prototype may prove that natural-language access is possible. It does not prove that the system is governed, reusable, explainable, or operable at scale; separate evaluation methods are needed for retrieval quality, grounding, and citation fidelity [7], [8].

2.3 From Assistants to Agentic Systems

Enterprise AI systems are expanding in scope. Early implementations were often limited to chatbots and FAQ-style assistants. The next wave introduced internal copilots that could summarize documents, answer policy questions, and synthesize information across repositories. The current direction is toward systems that can plan, retrieve, validate, call tools, interact with APIs, maintain intermediate state, and execute actions inside business workflows [15], [16], [17], [18].

This shift changes the architectural burden. A read-only assistant can hide weaknesses in retrieval, orchestration, state management, and policy enforcement for some time. An action-oriented system cannot. Once a system selects tools, calls enterprise APIs, drafts records, opens tickets, initiates approvals, or modifies downstream systems, the central question changes. It is no longer only whether the generated answer is correct. It is whether the system can act with the right permissions, within the right policy boundary, and with a reliable record of what it did and why.

A policy assistant illustrates the problem. In its first phase, it may be read-only and constrained to approved documents. It helps users locate guidance, summarize procedures, and answer routine questions. Encouraged by adoption, the organization may extend the assistant into workflow support: drafting incident summaries, retrieving supporting records, opening tickets, or initiating requests in operational systems. At that stage, weaknesses that were tolerable in the pilot become material. Tool permissions may be inconsistent. Retrieval logs may be incomplete. Prompt and workflow changes may not be versioned. The system may lack a clear control layer governing when it can move from retrieval and reasoning into action.

The same pattern appears in banking, compliance, internal operations, and employee copilots. A system may start by summarizing policy documents, regulatory guidance, or support knowledge. In a narrow pilot, it appears reliable because the corpus is curated and questions are predictable. As usage expands, users ask jurisdiction-specific questions, reference product exceptions, expect current guidance over superseded controls, and assume the assistant understands which sources are authoritative. Without lineage, effective dates, jurisdictional metadata, entitlement boundaries, and evaluation coverage, the assistant may produce fluent but operationally unsafe answers.

The point is not to avoid agentic systems. The point is that action-oriented AI makes platform responsibilities unavoidable. Tool invocation, workflow execution, permission enforcement, state management, versioning, auditability, and rollback behavior cannot be improvised at the application edge; tool-using and reasoning-acting systems make these controls part of the execution path [15], [16], [17], [18].

2.4 Gaps in Current Approaches

Many enterprise AI systems still struggle when they move from demonstration to production. In practice, the issue is usually not the absence of capable components. Modern stacks include strong foundation models, embedding models, vector stores, orchestration frameworks, catalogs, policy engines, governance tools, and observability platforms. The

harder problem is architectural: these components are often assembled into narrow pipelines without clear platform boundaries, operating controls, or lifecycle discipline.

Four gaps appear repeatedly.

First, many programs over-index on model selection. Teams compare model providers, prompt strategies, context-window sizes, and inference costs in detail, while giving less attention to retrieval quality, metadata discipline, authorization propagation, source authority, and evaluation coverage. This imbalance is understandable. Model behavior is visible at the interface. Platform behavior is not. But many failures that users experience as poor model performance begin upstream in the knowledge system.

Second, AI initiatives are often built outside the enterprise data and governance ecosystem. This can accelerate early delivery, but it creates long-term fragmentation: duplicated ingestion pipelines, inconsistent access-control enforcement, stale indexes, uneven metadata, and limited reuse across teams. The result is not an enterprise AI capability, but a collection of isolated assistants that solve similar problems in incompatible ways.

Third, early implementation choices often become long-term constraints. Managed services and high-level frameworks can be valuable, especially when teams need to move quickly. The risk is not using managed services. The risk is allowing application logic to become tightly coupled to a specific vector store, orchestration framework, model endpoint, or proprietary workflow abstraction. Without stable interfaces between layers, the cost of changing components later can exceed the benefit of initial acceleration.

Fourth, many systems are built as proofs of concept rather than operable products. Observability, evaluation, lifecycle management, policy enforcement, and release discipline are treated as later additions. In practice, they are the mechanisms that determine whether the system can improve safely after launch. Without them, teams can demonstrate capability but cannot reliably diagnose failures, compare versions, enforce policy, or manage quality as data sources, permissions, models, and user behavior change.

2.5 Positioning Against Related Architectural Patterns

The proposed reference architecture builds on existing architectural and technical traditions. It does not replace retrieval-augmented generation, data mesh, data fabric, lakehouse architecture, vector databases, or agentic workflow frameworks [1], [9]-[12], [15]-[18]. Each pattern addresses an important part of the enterprise AI problem. The issue is that none of them, on its own, defines the full operating model required for governed, retrieval-based enterprise AI systems.

The missing elements are usually not individual features. They are cross-layer responsibilities: ingestion contracts, metadata propagation, authorization-aware retrieval, source authority, index lifecycle management, orchestration controls, evaluation, observability, and application integration. GKS-5 positions these responsibilities inside a platform model so that they can be designed, governed, evaluated, and evolved deliberately.

Retrieval-augmented generation provides the grounding pattern most directly associated with enterprise AI [1]. It allows proprietary or frequently changing content to remain outside the model while still informing generated responses. This makes RAG essential for many knowledge-access use cases. However, RAG does not specify how content should be ingested, governed, indexed, authorized, evaluated, monitored, or retired. In this paper, RAG is treated as an interaction pattern within a broader Governed Knowledge System, not as the architecture itself.

Data mesh contributes principles of domain ownership and decentralized accountability [9], [10], [11]. These principles are relevant because enterprise knowledge quality depends on accountable source owners and well-defined domains. However, data mesh does not prescribe semantic indexing, hybrid retrieval, context assembly, model orchestration, or controlled agentic execution. GKS-5 can use data mesh principles for ownership and stewardship while extending them into retrieval and orchestration.

Data fabric contributes metadata-driven integration, interoperability, and governance across distributed environments [11]. These capabilities are important where enterprise knowledge spans repositories, SaaS systems, operational platforms, collaboration tools, and analytical stores. However, data fabric is usually oriented toward data access and integration, not retrieval-time reasoning, dynamic context construction, prompt-time grounding, or model-mediated workflows. In this paper, data fabric capabilities are treated as enabling infrastructure for knowledge integration.

Lakehouse architectures provide durable storage, scalable processing, lineage, and governance over structured and unstructured data [12]. They are valuable foundations for preserving raw content, supporting reprocessing, and maintaining analytical consistency. However, a lakehouse does not automatically produce retrieval-ready chunks, embeddings, hybrid indexes, ranking strategies, or orchestration controls. In GKS-5, lakehouse capabilities support the storage and processing foundation, but they do not replace the retrieval and orchestration layers.

Vector databases and similarity search systems provide essential primitives for semantic retrieval and large-scale nearest-neighbor search [2], [6]. They are useful for finding conceptually related passages in unstructured and semi-structured content. In enterprise settings, however, semantic similarity is only one retrieval signal. Many questions require exact matching, metadata filtering, temporal constraints, source authority, access-control enforcement, deduplication, and citation-aware context assembly. Vector databases are therefore components of the storage/indexing and retrieval layers, not the enterprise AI architecture.

Agentic workflow frameworks support planning, tool use, memory, multi-step execution, and interaction with external systems [15], [16], [17], [18]. They become important as enterprise AI systems move from read-only assistance toward action-oriented workflows. However, agent frameworks often assume that data access, retrieval quality, permissions, observability, and policy controls already exist. In enterprise environments, that assumption is risky. GKS-5 treats agentic workflows as consumers of a governed knowledge platform rather than substitutes for one.

Table 1. Relationship between existing architectural patterns and GKS-5.

Pattern	Primary contribution	Limitation for enterprise AI	Role in GKS-5
Retrieval-Augmented Generation	Grounds responses in external knowledge.	Does not define governance, authorization, evaluation, or lifecycle controls.	Grounding pattern inside the broader platform.
Data Mesh	Domain ownership and accountability.	Does not specify semantic indexing, context assembly, orchestration, or agents.	Ownership and stewardship principle.
Data Fabric	Metadata-driven integration and governance.	Usually oriented toward integration rather than retrieval-time reasoning or model-mediated workflows.	Enabling infrastructure for knowledge integration.
Lakehouse	Scalable storage, lineage, and analytics.	Does not automatically provide retrieval-ready indexing or orchestration.	Storage and reprocessing foundation.
Vector Databases	Semantic similarity search.	Insufficient for exact matching, policy, freshness, and authority.	Component of indexing and retrieval layers.
Agentic Frameworks	Tool use, planning, and multi-step execution.	Often assume governed data, permissions, and observability already exist.	Consumer of the governed platform.

3. Problem Statement

Enterprises do not struggle with AI knowledge systems because one component is missing. They struggle because production systems must operate across several hard boundaries at once: fragmented knowledge sources, probabilistic retrieval, changing permissions, uneven metadata, cost constraints, security obligations, and rising expectations for what AI systems should be able to do.

The first challenge is knowledge fragmentation. Enterprise knowledge is distributed across document repositories, relational systems, object stores, SaaS applications, ticketing platforms, collaboration tools, APIs, and domain-specific operational systems. These sources differ not only in format, but also in ownership, authority, update frequency, metadata quality, retention rules, and access semantics. Building a useful AI system over this landscape requires more than indexing content. It requires preserving the distinctions that make knowledge trustworthy: provenance, permissions, source authority, business context, and domain ownership.

The second challenge is freshness. Many enterprise AI systems are expected to answer questions about information that changes continuously, while ingestion and indexing pipelines often remain periodic or batch-oriented. More frequent ingestion can improve relevance and user trust, but it also increases cost, operational complexity, and failure surface area. Not every source needs the same update cadence. A quarterly policy manual, an active incident record, a

customer case, and a pricing table should not be synchronized in the same way. Yet many early systems apply a uniform ingestion pattern because it is simpler to build.

The third challenge is cost visibility. Enterprise AI cost is often discussed as if it were primarily a model-inference problem. In practice, cost is distributed across the platform: connectors, parsing, extraction, transformation, storage, indexing, embeddings, retrieval, ranking, caching, orchestration, guardrails, monitoring, evaluation, and human review. A system that appears inexpensive in a pilot can become costly at scale if retrieval is inefficient, indexes are rebuilt unnecessarily, context windows expand without discipline, or users must manually verify unreliable outputs. Optimizing only for token cost misses the economics of the knowledge platform.

The fourth challenge is governance. Access control, lineage, retention, auditability, and sensitive data handling become more dynamic when AI is added to the architecture. Retrieval-based systems assemble context at query time, often from multiple sources. Agentic systems may retrieve, reason, invoke tools, and initiate actions across several enterprise systems in a single workflow. This creates policy questions that are harder to inspect than conventional analytical queries: which sources influenced the answer, whether the user was entitled to them, whether stale or restricted content was excluded, and whether a tool was invoked under the correct authority.

The fifth challenge is adaptability. Enterprise AI is changing quickly. Model providers, smaller specialist models, multimodal capabilities, retrieval strategies, orchestration frameworks, and agentic workflow patterns continue to evolve. Organizations that bind their applications too tightly to one model endpoint, vector store, retrieval strategy, or managed orchestration layer may gain speed in the short term but lose flexibility later. In a fast-moving ecosystem, architecture must preserve credible paths for substitution, extension, and migration.

These challenges are connected. Fragmented sources make freshness harder to manage. Poor metadata weakens retrieval and governance. Weak observability hides cost and quality problems. Tight coupling makes it harder to adopt better models, retrieval methods, or deployment patterns. The central problem is therefore not how to connect a language model to enterprise data. It is how to design a governed platform that integrates ingestion, storage, retrieval, reasoning, and application delivery without collapsing into brittle, one-off pipelines.

This paper summarizes that problem as knowledge integrity drift.

Knowledge integrity drift is the degradation over time of alignment between enterprise source systems, metadata, permissions, indexes, retrieval behavior, orchestration logic, and the outputs users see. The system may still appear operational: documents are indexed, answers are generated, and users receive fluent responses. Yet the knowledge path becomes less trustworthy. Sources may be stale. Metadata may be incomplete. Permissions may be outdated. Retrieval rankings may favor informal or superseded content. Prompts may change without evaluation. Citations may no longer support the claims they accompany.

Knowledge integrity drift explains why many enterprise AI systems perform well in pilots but degrade after expansion. In a pilot, the corpus is curated, permissions are simple, and evaluation is informal. Production introduces changing sources, domain-specific metadata, conflicting documents, evolving user behavior, and policy constraints. Unless the platform actively preserves knowledge integrity across layers, apparent model failures will continue to emerge from platform drift.

4. Reference Architecture

This section introduces GKS-5, a five-layer reference model for Governed Knowledge Systems. The model is not intended to prescribe a single product stack, cloud provider, deployment pattern, or vendor path. Its purpose is to separate the responsibilities that are often collapsed in early pilots: ingestion, storage and semantic indexing, retrieval, model orchestration, and application integration.

Treating these responsibilities as distinct layers makes enterprise AI systems easier to govern, evaluate, tune, and evolve. A team should be able to improve chunking without changing the user interface, add lexical search without replacing the orchestration layer, introduce a new model provider without rebuilding ingestion, or expose the same governed retrieval capability through a chat interface, workflow system, and API.

GKS-5 should be understood as a structural model. The five layers define where core platform responsibilities reside. They do not, by themselves, make a system ready for production. Production readiness also requires cross-layer

controls for metadata, authorization, freshness, observability, evaluation, and versioning. The layers define the architecture. The controls preserve knowledge integrity across it.

4.1 Design Principles

GKS-5 is guided by five design principles.

Separation of concerns

Ingestion, storage, retrieval, orchestration, and application integration should be treated as separate architectural responsibilities, even when a small number of services initially implement several of them. This separation prevents early implementation choices from hardening into platform constraints. It also allows teams to improve retrieval, change models, modify ingestion pipelines, or introduce new user experiences without redesigning the entire system.

Governance by construction

Governance should not be added after the assistant works. Access metadata, provenance, retention rules, source authority, sensitivity labels, and audit requirements should move through the platform from the beginning. If these attributes are lost during ingestion or indexing, they are difficult to reconstruct later and may already have influenced retrieval or generation. A Governed Knowledge System carries policy-relevant context alongside content throughout the knowledge path.

Hybrid retrieval by default

Enterprise knowledge is too heterogeneous for a single retrieval method to perform well across all use cases. Semantic retrieval is useful for conceptual similarity, but many enterprise questions depend on exact identifiers, policy clauses, product codes, dates, jurisdictions, or source authority. The platform should support semantic search, lexical search, metadata-aware filtering, ranking, and re-ranking as complementary mechanisms rather than treating vector search as the retrieval layer.

Vendor-neutral abstraction

Managed services can accelerate delivery and reduce operational burden, especially in early phases. The architectural risk is not the use of managed services. The risk is allowing product-specific assumptions to define the platform. Interfaces between layers should be explicit enough that organizations can change models, vector stores, orchestration frameworks, deployment environments, or governance tools without rewriting every application built on top of the platform.

Extensibility toward action

Enterprise AI systems are moving from answering questions to supporting workflows. The platform should therefore support not only search and summarization, but also multi-step reasoning, tool invocation, API interaction, approval flows, and controlled execution. This does not mean every system should become agentic immediately. It means the architecture should leave room for action-oriented use cases without bypassing retrieval controls, authorization, observability, or auditability.

4.2 Architecture Overview

GKS-5 consists of five logical layers:

Data Ingestion

Storage and Semantic Indexing

Retrieval

Model Orchestration

Application Integration

GKS-5: A Layered Architecture for Governed Knowledge Systems

Separating ingestion, storage, retrieval, orchestration, and application integration

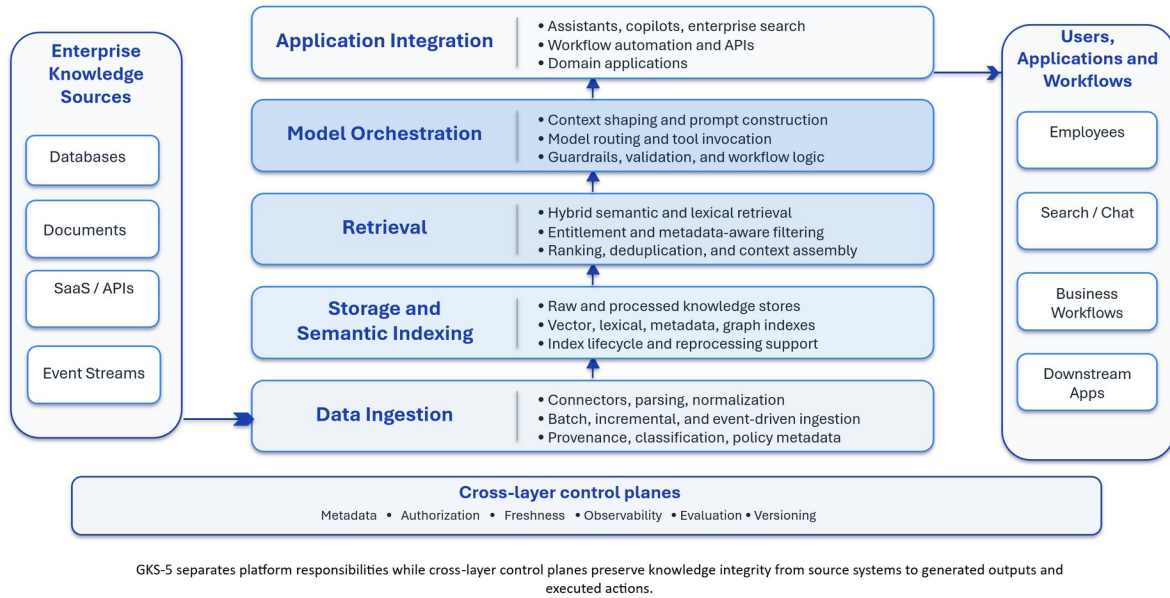


Figure 2. GKS-5 platform-centric reference architecture for Governed Knowledge Systems. The five layers separate platform responsibilities while the controls preserve knowledge integrity from source systems to generated outputs and executed actions.

Table 2. GKS-5 layers and primary responsibilities.

Layer	Primary responsibilities	Production concern
Data ingestion	Connectors, parsing, normalization, classification, provenance capture, policy metadata, source identifiers, ownership, and freshness signals.	Losing provenance, permissions, or source context during ingestion weakens every downstream layer.
Storage and semantic indexing	Raw and processed stores; vector, lexical, metadata, and graph indexes; index lifecycle and reprocessing support.	Embeddings alone do not preserve authority, freshness, exact identifiers, or access semantics.
Retrieval	Query understanding, policy-aware scoping, hybrid retrieval, ranking, deduplication, source validation, and context assembly.	Retrieval must be part of the security and quality boundary, not a single vector-search call.
Model orchestration	Context shaping, prompt construction, model routing, validation, refusal behavior, tool invocation, workflow logic, and fallback handling.	Reasoning, policy, and workflow behavior must be versioned, observable, and testable.
Application integration	Assistants, enterprise search, copilots, APIs, workflow integration, reporting surfaces, and domain applications.	The interface should expose platform capabilities without trapping governance logic in one application.

The layers are logical responsibilities, not necessarily separate products or services. In a small deployment, one managed platform may implement several of them. In a larger enterprise or regulated environment, each layer may be implemented by multiple services across cloud, on-premises, or sovereign infrastructure. Physical separation is not the goal. The goal is that the responsibilities remain visible, governable, and replaceable.

Each layer should expose clear interfaces to the layers around it. The goal is not to eliminate coupling entirely. Real systems always contain some coupling, especially where performance, security, latency, or operational constraints matter. The goal is to prevent a local implementation choice from becoming a platform constraint. An organization should be able to improve chunking without changing the user interface, add lexical search without replacing orchestration, introduce a new model provider without rebuilding ingestion, or expose the same governed retrieval capability through a chat interface, workflow system, and API.

This separation is what turns a successful pilot into reusable platform capability. In many early systems, ingestion, retrieval, prompt construction, application logic, and evaluation are collapsed into one pipeline. That can be fast, but

it makes the system difficult to govern and difficult to evolve. GKS-5 separates these responsibilities so that quality, policy, cost, and operational behavior can be managed at the right layer.

The five layers define the structural foundation of a Governed Knowledge System. They do not, by themselves, make the system ready for production. A system can contain all five layers and still remain fragile if metadata is lost between layers, permissions are enforced only at the interface, freshness is invisible to retrieval, or prompts and indexes change without version control. For that reason, GKS-5 must be paired with cross-layer controls for metadata, authorization, freshness, observability, evaluation, and versioning. The layers define the platform structure. The controls preserve knowledge integrity across it.

4.3 Data Ingestion Layer

The Data Ingestion Layer acquires, normalizes, enriches, and prepares enterprise knowledge for downstream use. Sources may include relational databases, object stores, content management systems, collaboration platforms, event streams, APIs, SaaS applications, ticketing systems, and domain-specific operational systems.

Its role is broader than extraction. In production systems, ingestion is where long-term knowledge quality is often created or lost. This is the stage where the platform parses documents, standardizes formats, captures provenance, preserves source identifiers, records ownership, attaches classifications, and prepares content for indexing. Decisions made here determine whether later layers can retrieve the right evidence, enforce the right permissions, distinguish current from obsolete material, and explain where an answer came from.

For unstructured content, ingestion may include parsing PDFs, office documents, web pages, messages, tables, images, and attachments. It may involve language detection, optical character recognition where necessary, document segmentation, table extraction, boilerplate removal, and chunking strategies aligned to retrieval behavior. Chunking is not a mechanical preprocessing step. Poor chunking can make relevant content difficult to retrieve. Overly large chunks can dilute relevance. Overly small chunks can remove the context needed for interpretation. Ingestion design must therefore reflect how the knowledge will eventually be searched, ranked, and used.

For structured and semi-structured systems, ingestion should preserve business keys, timestamps, relationships, source-system identifiers, and domain metadata. These attributes allow downstream components to reconstruct business context rather than treating every field as isolated text. A support ticket, for example, may only be meaningful when linked to product, customer, severity, time period, entitlement, and resolution state. Losing that context weakens both retrieval quality and governance.

The ingestion layer should also support different freshness patterns. Slow-moving repositories may be synchronized periodically. Frequently changing operational systems may require incremental ingestion, event-driven updates, or near-real-time indexing. Permission changes may need to propagate faster than content changes. Freshness should therefore be designed deliberately. Treating every source as real-time is expensive and often unnecessary. Treating every source as batch-oriented creates stale and untrustworthy systems.

Governance begins in this layer. Sensitive content should be identified early. Ownership, document classification, retention metadata, jurisdictional constraints, lineage identifiers, source authority, and access attributes should travel with the content into downstream storage, indexing, retrieval, and orchestration. Teams often try to infer these properties later, but late inference is brittle and incomplete. When retrieval fails in production, weak ingestion metadata is often the hidden cause.

A strong ingestion layer does more than move data. It converts fragmented enterprise sources into usable, traceable, policy-aware knowledge assets. Without that foundation, later improvements in retrieval, prompting, or model selection can only compensate partially.

4.4 Storage and Semantic Indexing Layer

The Storage and Semantic Indexing Layer provides the durable foundation for enterprise AI knowledge systems. It stores both original source content and the processed artifacts required for retrieval, ranking, governance, and reprocessing. This is where enterprise knowledge becomes operationally usable by AI applications.

Most production architectures require more than one storage pattern. Raw content is typically retained in an object store, lakehouse, content repository, or comparable durable layer. This preserves fidelity to the source and allows the platform to reprocess content as parsers, chunking strategies, embedding models, or governance requirements change.

Processed artifacts may include normalized text, extracted tables, document metadata, chunk-level metadata, access-control projections, embeddings, keyword indexes, graph relationships, and references back to the original source.

Maintaining both raw and processed forms is not unnecessary duplication. The two forms serve different purposes. Raw storage supports traceability, auditability, and future reprocessing. Processed representations support low-latency retrieval, search efficiency, ranking, filtering, and application performance. Without raw preservation, the system becomes difficult to audit or improve. Without processed representations, retrieval becomes too slow, expensive, or inconsistent for interactive use.

Semantic indexing is a central responsibility of this layer. Unstructured and semi-structured content is transformed into vector representations that support meaning-based retrieval [2], [6]. This is essential because many enterprise questions do not match the exact wording of source material. However, semantic indexing alone is insufficient. Enterprise knowledge contains product codes, policy clauses, legal terms, account names, ticket identifiers, control numbers, regulatory references, and operational abbreviations. These often require exact or near-exact matching, where lexical retrieval methods such as BM25 remain important [3]. A system that relies only on vector similarity may retrieve text that is conceptually plausible but operationally wrong.

For that reason, this layer should support a multi-index foundation. Vector indexes, lexical indexes, metadata stores, graph relationships, and structured filters should be able to coexist. Metadata is especially important because retrieval often needs to be scoped by role, geography, business unit, document type, confidentiality level, jurisdiction, time period, source authority, or effective date. In enterprise environments, the question is rarely only "What is semantically similar?" It is also "What is current, authoritative, permitted, and relevant for this user?"

This layer must also manage index lifecycle and update policy. As corpora grow, re-embedding or rebuilding entire indexes becomes expensive and operationally disruptive. A stronger design distinguishes between different types of change. A content change may require re-parsing, re-chunking, and re-embedding. A metadata change may require only index updates. A permission change may need to propagate quickly even when the underlying content is unchanged. Treating all changes the same leads either to excessive cost or stale behavior.

Partitioning and isolation should be explicit design decisions. Some corpora may need to be separated by tenant, business unit, jurisdiction, sensitivity, or deployment environment. Other corpora may be shared across applications but filtered dynamically at retrieval time. These choices affect cost, latency, security, portability, and governance. They should not be inherited accidentally from the first vector database or search service used in a pilot.

The storage and semantic indexing layer therefore does more than hold embeddings. It preserves enterprise knowledge in forms that are traceable, searchable, governable, and efficient. Vector search is an important primitive, but it is not a complete indexing strategy. The quality of this layer determines whether downstream retrieval can reliably find the right evidence under the right constraints.

4.5 Retrieval Layer

The first version of an enterprise assistant often performs well in a demonstration and poorly in production for the same reason: the demonstration hides the boundary conditions of retrieval. The corpus is smaller, the questions are more predictable, permissions are simpler, and the content is usually curated. Production removes those advantages.

The Retrieval Layer is the bridge between enterprise knowledge and model reasoning. In many enterprise systems, retrieval quality can have as much influence on perceived answer quality as model selection. If the right evidence is not retrieved, no prompt can reliably recover it. If stale, unauthorized, duplicated, or low-authority evidence is retrieved, the model may produce an answer that sounds fluent but is unsafe, unsupported, or wrong.

Retrieval begins with query understanding. Enterprise questions are often ambiguous, incomplete, or mixed in intent. A user may ask for a policy explanation while also including a product code, a region, a date, and an implied business context. Another user may ask a short question that only makes sense inside a specific role, workflow, or system. Effective retrieval therefore requires more than embedding the query and searching a vector index; dense retrieval is useful, but it is only one part of a broader retrieval strategy [2], [6]. The platform may need to normalize terminology, classify intent, extract entities, infer filters, rewrite the query, identify the relevant domain, or route the request across different retrieval paths.

Enterprise retrieval also has to combine multiple retrieval mechanisms, including dense retrieval, lexical retrieval, contextualized re-ranking, and learning-to-rank methods [2], [3], [4], [5]. Semantic retrieval is useful when the user's

wording differs from the source. Lexical retrieval is necessary when exact terms, identifiers, clauses, product codes, ticket numbers, or regulatory references matter. Metadata-aware filtering is required when results must be scoped by permission, source, date, business unit, jurisdiction, document type, sensitivity, or source authority. Re-ranking, deduplication, and context assembly determine which evidence the model ultimately sees.

These mechanisms are not interchangeable. Each contributes differently to recall, precision, latency, explainability, and cost. Some architectures use parallel hybrid search and result fusion. Others use staged retrieval, query routing, or domain-specific re-rankers, consistent with broader information retrieval and learning-to-rank approaches [4], [5]. The design can vary by use case. What should not vary is the assumption that enterprise data and enterprise questions are heterogeneous.

Authorization must be enforced in the retrieval layer, not merely at the user interface. Retrieving broadly and redacting later is unsafe because retrieved content can influence generation even if it is not displayed. Access controls, entitlements, source restrictions, tenant boundaries, confidentiality labels, and jurisdictional rules should shape candidate selection before context is assembled and passed to the model. In GKS-5, the retrieval layer is part of the security boundary.

Ranking and context assembly are equally important. It is not enough to find relevant documents. The system must select the right passages, preserve provenance, remove duplicates, resolve conflicts where possible, and present the model with context it can actually use. Chunk boundaries, passage ordering, source authority, and context packing often matter more than prompt wording. A well-written prompt cannot compensate for missing evidence, misleading context, or unauthorized retrieval.

A production retrieval layer should also account for source authority. Enterprise corpora often contain multiple versions of similar knowledge: official policies, draft documents, working notes, wiki summaries, emails, tickets, regional exceptions, and obsolete files. Semantic similarity alone cannot determine which source should be trusted. A retrieved passage may be relevant but non-authoritative. For that reason, retrieval ranking should combine relevance with authority, freshness, metadata fit, and permission validity.

The permission term is not intended to replace pre-retrieval authorization enforcement; rather, it represents permission validity for candidates that have already passed policy-based eligibility checks. This expression is consistent with the broader information-retrieval principle that ranking can combine multiple relevance and quality signals rather than relying on a single similarity score [3], [5]. It is not intended as a universal scoring formula. Its purpose is to make the design principle explicit: enterprise retrieval should not optimize only for similarity. It should optimize for evidence that is permitted, current, authoritative, and contextually appropriate.

A simplified ranking model can be expressed as:

$$\begin{aligned} R(q,d,u) = & w_1 S_{sem}(q,d) + w_2 S_{lex}(q,d) + w_3 S_{meta}(d,u) \\ & + w_4 S_{fresh}(d) + w_5 S_{auth}(d) + w_6 S_{perm}(d,u) \\ & - w_7 P_{dup}(d) - w_8 P_{super}(d) \end{aligned}$$

where q is the user query, d is a candidate document or passage, and u is the user context. S_{sem} represents semantic relevance, S_{lex} represents lexical match quality, S_{meta} represents metadata fit, S_{fresh} represents freshness, S_{auth} represents source authority, and S_{perm} represents permission validity. P_{dup} and P_{super} represent duplication and supersession penalties. The weights w_1 through w_8 can be tuned by use case, domain, risk level, and retrieval strategy.

The retrieval layer should expose diagnostic information. Teams need to know which query interpretation was used, which filters were applied, which sources were searched, why certain passages were ranked highly, and which evidence was ultimately passed to the model. Without this visibility, failures are difficult to diagnose and are often misclassified as model problems.

Enterprise retrieval is therefore not a single vector search operation. It is a governed sequence of query interpretation, authorization-aware candidate selection, hybrid search, ranking, deduplication, source validation, and context assembly. Treating retrieval as a first-class platform layer is one of the main differences between a useful pilot and a ready for production Governed Knowledge System.

4.6 Model Orchestration Layer

The Model Orchestration Layer determines how retrieved evidence becomes a useful, safe, and inspectable output. It coordinates context shaping, prompt construction, model routing, tool invocation, structured output generation, fallback behavior, response validation, and evaluation hooks. In a prototype, orchestration may be little more than a prompt template wrapped around a model call. In a production platform, it becomes the control layer that governs how reasoning is performed.

One of its most important responsibilities is context shaping. More context is not automatically better context. Large context windows can make it tempting to pass too much downstream, but doing so increases cost, latency, and ambiguity. Models perform better when they receive evidence that is relevant, current, well ordered, and framed for the task. The orchestration layer decides which retrieved passages to include, when to summarize or compress evidence, how to represent citations, how to handle conflicting sources, and when the available evidence is too weak to support a confident answer.

This layer also provides an abstraction boundary over model providers and inference patterns. Production systems rarely need one model for every task. A smaller model may be sufficient for classification, extraction, routing, or guardrail checks. A more capable model may be reserved for complex reasoning, synthesis, or high-value workflows. Some requests require deterministic structured output. Others require exploratory summarization. Treating orchestration as a platform layer allows these decisions to evolve without rewriting ingestion, retrieval, or application code.

Orchestration also manages the relationship between model behavior and enterprise policy. A model may need to refuse an answer when evidence is insufficient, cite only approved sources, prefer authoritative documents over informal notes, or avoid producing advice outside a permitted domain. These controls should not depend only on prompt wording. They should be enforced through orchestration logic, validation rules, policy checks, and observable decision paths.

As enterprise AI systems become more agentic, orchestration expands from response generation to controlled execution. The system may request additional retrieval, call enterprise APIs, invoke tools, draft records, open tickets, or propose actions inside a workflow. These steps require more than model capability. They require permission checks, tool-scoping rules, approval policies, state management, rollback behavior, and audit trails. This is where the difference between a demonstration and a platform becomes visible: in a demo, orchestration often hides inside the application; in a platform, orchestration becomes part of the enterprise control surface.

This layer should also support versioning and evaluation. Prompt templates, routing logic, tool permissions, model choices, fallback paths, and validation rules should be treated as versioned operational assets. Enterprises need to compare versions, detect regressions, inspect failures, and understand how changes affect quality, cost, latency, and risk. Without this discipline, orchestration becomes an invisible source of drift.

A mature orchestration layer does not merely call a model. It decides how evidence is used, which model or tool path is appropriate, what policy constraints apply, how outputs are validated, and how the system behaves when confidence is low. It is the layer where retrieval, reasoning, policy, and operational control come together.

4.7 Application Integration Layer

The Application Integration Layer exposes platform capabilities to users, applications, and business processes. It may appear as a chat interface, enterprise search experience, embedded copilot, service desk assistant, workflow integration, API, reporting tool, or domain-specific application. This is where users experience the value of the platform, but it should not be confused with the platform itself.

A common enterprise mistake is to treat interface quality for system maturity. A polished chat interface can create the appearance of readiness even when the underlying platform lacks reliable ingestion, authorization-aware retrieval, observability, evaluation, or lifecycle management. A polished interface does not fix the underlying platform. It exposes the platform's strengths and weaknesses to users.

Application integration should support both human-facing and machine-facing patterns. Human-facing experiences need clarity, usability, source citations, provenance indicators, confidence cues, feedback mechanisms, and graceful handling of uncertainty. Users need to understand not only the answer, but where it came from, whether it is current, and what limits apply to it. Trust is built through transparency and restraint, not only through fluent responses.

Machine-facing integrations have different requirements. They depend on stable APIs, structured outputs, idempotent behavior, predictable error handling, rate limits, authentication, and operational reliability. A workflow system consuming an AI-generated output may need a JSON object, confidence signal, citation map, refusal reason, or approval status rather than a conversational answer. A mature platform should support these integration modes without duplicating the knowledge stack for each application.

This layer may also manage session behavior, personalization, rate limits, interaction logging, feedback capture, and user experience policies. These capabilities are useful, but they should extend platform capabilities rather than compensate for weak foundations. User feedback can improve retrieval and evaluation, but it cannot substitute for source authority, metadata quality, freshness management, or access-control propagation.

The application layer should therefore remain thin enough to avoid trapping platform logic inside individual interfaces. A chat assistant, search page, workflow integration, and API endpoint may present different experiences, but they should rely on shared ingestion, retrieval, orchestration, governance, and observability services. This separation is what allows enterprise AI to mature from isolated applications into reusable platform capability.

4.8 Worked example: a stale policy with a regional exception

Consider a policy assistant that answers questions about employee travel reimbursement. In a pilot, the corpus may contain one approved policy PDF. Retrieval is easy, and the assistant appears accurate. After expansion, the corpus may include an old policy copy in a shared drive, a new policy in a records system, a regional exception in a collaboration site, and an informal FAQ that uses simpler language than the official policy. A pure semantic search path may retrieve the FAQ first because it resembles the user question. A pure lexical path may retrieve the old policy because it contains the exact clause number. Neither result is enough.

In GKS-5, the same question should be routed through eligibility, freshness, authority, and retrieval checks before evidence reaches the model. The user entitlement determines which sources can be searched. Effective-date metadata and supersession status reduce the rank of older policy copies. A source-authority signal favors the approved policy over the FAQ. Lexical retrieval preserves exact clause matching, while semantic retrieval captures paraphrased user intent. The final context package should include the official policy passage, the applicable regional exception if the user is in scope, and enough provenance for the answer and citation to be audited later.

A governed retrieval sequence from query interpretation to context assembly for model orchestration

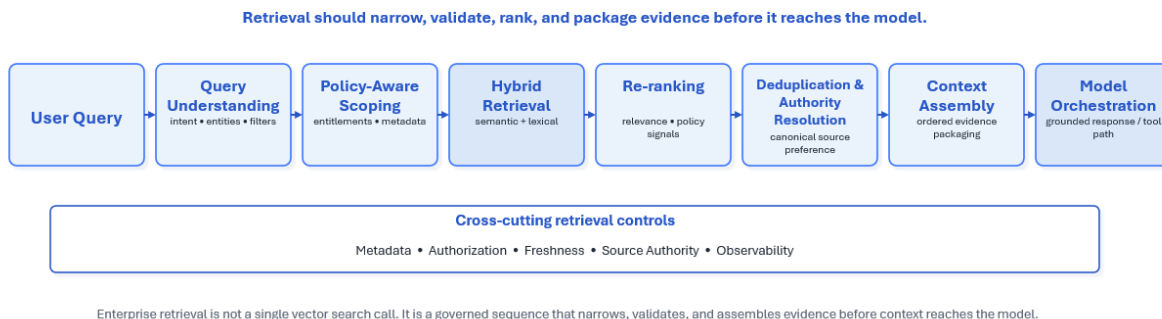


Figure 3. Governed enterprise retrieval flow. Retrieval begins with query interpretation and policy-aware scoping, proceeds through hybrid search, ranking, deduplication, and authority resolution, and ends with context assembly for model orchestration.

5. Cross-Layer Control Planes

The five GKS-5 layers define the structural responsibilities of a Governed Knowledge System. Production readiness, however, depends on capabilities that cut across those layers. The section describes these capabilities as controls.

A control plane is a cross-layer mechanism that governs how knowledge, policy, system behavior, and operational evidence move through the platform. Without controls, the five layers can still collapse into a brittle pipeline. Content may be ingested but not trusted, indexed but not current, retrieved but not authorized, generated but not traceable, or deployed but not evaluable.

This distinction is central to GKS-5. The layers define where platform responsibilities reside. The controls define how knowledge integrity is preserved across those responsibilities.

This paper identifies six controls required for production-grade Governed Knowledge Systems.

These controls are not separate products. They are architectural obligations. A given implementation may realize them through metadata catalogs, policy engines, tracing systems, evaluation frameworks, workflow registries, CI/CD pipelines, or platform-specific services, including observability and policy-enforcement frameworks such as OpenTelemetry and Open Policy Agent [22], [23]. What matters is that the capability persists across the full knowledge path.

For example, metadata captured during ingestion must remain available during indexing, retrieval, context assembly, generation, citation, and audit. Authorization must shape candidate retrieval before model context is assembled, not merely redact output after generation, consistent with modern security-control and zero-trust principles [20], [21]. Freshness must be visible not only to ingestion jobs, but also to retrieval ranking and orchestration decisions. Evaluation must test individual layers as well as end-to-end answer quality.

GKS-5 therefore consists of two linked parts: a five-layer structural model and six cross-layer controls. The layers define the platform structure. The controls preserve trust, governance, and operational discipline across that structure.

Table 3. Cross-layer controls in GKS-5.

Control	Function	Risk mitigated
Metadata	Preserves source, ownership, document type, effective date, jurisdiction, sensitivity, and authority attributes across layers.	Loss of provenance, weak ranking, poor citations, and confusion between authoritative and informal content.
Authorization	Enforces user, role, tenant, source, document, and action-level permissions before retrieval, context assembly, and tool execution.	Exposure of restricted content or unauthorized downstream actions.
Freshness	Tracks source update time, ingestion time, index time, staleness tolerance, and supersession status.	Reliance on stale, obsolete, or superseded information.
Observability	Traces behavior from ingestion through retrieval, orchestration, response generation, and action execution.	Inability to diagnose failures, audit behavior, or explain outputs.
Evaluation	Measures retrieval relevance, grounding, citation support, policy compliance, latency, cost, and task success.	Demos that appear successful but fail under production conditions.
Versioning	Versions prompts, retrieval parameters, indexes, embedding models, policies, tools, and orchestration flows.	Uncontrolled drift, poor reproducibility, and inability to compare or roll back behavior.

6. Failure-Mode Taxonomy for Enterprise AI Knowledge Systems

Enterprise AI failures are often reported at the interface level. Users describe hallucinations, wrong answers, weak citations, slow responses, or unsafe recommendations. These symptoms matter, but they rarely identify where the failure originated. A stale answer, for example, may result from delayed ingestion, missing freshness metadata, poor retrieval ranking, weak context assembly, or orchestration logic that failed to detect insufficient evidence.

A failure-mode taxonomy helps separate user-visible symptoms from platform root causes. The distinction matters because changing the model or prompt will not reliably fix failures caused by stale indexes, weak metadata, missing authorization filters, poor ranking, or inadequate evaluation coverage. In GKS-5, failures should be diagnosed by identifying the layer and control plane where knowledge integrity broke down.

This taxonomy reframes many apparent model failures as knowledge-system failures. A stronger model may reduce some surface-level errors, but it cannot reliably compensate for missing provenance, stale indexes, unauthorized

context, weak retrieval, or untested orchestration changes. The taxonomy supports a practical diagnostic discipline: before changing the model, teams should identify which layer and control plane produced the failure.

The taxonomy also provides the basis for readiness assessment. If the same failure modes appear repeatedly, the issue is not only incident-level quality. It is architectural maturity.

Table 4. Failure-mode taxonomy for Governed Knowledge Systems.

Failure mode	User-visible symptom	Likely root cause	Mitigation
Stale answer	Answer relies on outdated or superseded guidance.	Index not refreshed; freshness metadata missing or unused.	Freshness classes, index timestamps, supersession metadata, freshness-aware retrieval.
Unauthorized grounding	Answer appears influenced by content the user should not access.	Authorization applied after retrieval or only at interface.	Pre-retrieval authorization filtering and context-level entitlement enforcement.
Authority inversion	Informal wiki, draft, or working note outranks approved policy.	Source authority missing, inaccurate, or unused in ranking.	Authority scoring, canonical source tagging, deduplication, source validation.
Semantic overmatch	Retrieved passage is conceptually similar but operationally wrong.	Vector search used without lexical, metadata, temporal, or authority constraints.	Hybrid retrieval, metadata filters, exact-match paths, re-ranking.
Citation mismatch	Citation does not support the generated claim.	Context assembly or generation separates claims from supporting evidence.	Citation-aware context packing and citation fidelity evaluation.
Permission drift	Recent permission changes are not reflected in retrieval.	Entitlement updates lag behind content indexes.	Separate permission update path and fast entitlement propagation.
Evaluation blind spot	System passes demos but fails in production.	Test set does not cover freshness, permissions, authority, conflict, or edge cases.	Layered offline and online evaluation with production-like test cases.
Agentic overreach	System initiates or recommends an unsafe action.	Tool access not scoped by user, task, risk, or policy.	Tool permission policies, approval gates, action audit trails, rollback paths.

7. Governed Knowledge System Readiness Score

A reference architecture is useful only if organizations can assess whether their implementation satisfies its operational intent. This paper therefore introduces a Governed Knowledge System Readiness Score: a practical assessment instrument for determining whether an enterprise AI knowledge system is still a prototype, ready for a controlled pilot, suitable for production, or mature enough to support controlled agentic workflows.

The score is not intended to replace security review, compliance assessment, red teaming, or system testing. It provides a structured way to identify production-readiness gaps across the layers and controls of GKS-5.

Each dimension is scored from 0 to 5:

0 indicates that the capability is absent or ad hoc.

3 indicates that the capability is partially governed but incomplete.

5 indicates that the capability is automated, governed, observable, and integrated into operational release management.

A simple readiness score can be calculated as the average of the ten dimension scores. Organizations may also weight dimensions differently depending on regulatory exposure, workflow criticality, data sensitivity, or deployment environment.

Readiness interpretation

The score is most useful before scaling a pilot. A system may appear successful in a narrow demonstration while scoring poorly on authorization fidelity, freshness management, evaluation maturity, or operational traceability. Those weaknesses indicate that the system is not yet ready for broad enterprise deployment, even if its answers appear fluent.

The readiness score should also guide evaluation design. Low-scoring dimensions should become explicit test areas. A system with weak freshness management should be tested against superseded documents. A system with weak authorization fidelity should be tested against restricted sources. A system with limited orchestration governance should be tested against model-route changes, tool-use boundaries, and fallback behavior.

Table 5. Governed Knowledge System Readiness Score.

Dimension	0: Ad hoc/absent	3: Partially governed	5: Production-grade
Ingestion coverage	Manual upload or narrow corpus.	Multiple sources with partial automation.	Priority sources tracked with coverage, failure, and status metrics.
Metadata integrity	Minimal document-level metadata.	Source, owner, and date retained for some content.	Chunk-level provenance, ownership, effective date, authority, sensitivity, and jurisdiction.
Authorization fidelity	UI-level or post-generation filtering.	Partial role, source, or group filters.	Pre-retrieval and pre-tool authorization with auditable policy decisions.
Freshness management	No explicit freshness tracking.	Scheduled re-indexing for selected sources.	Source-specific freshness classes, index timestamps, supersession tracking, and staleness-aware retrieval.
Retrieval robustness	Basic vector search.	Hybrid search with limited metadata filtering.	Hybrid retrieval, re-ranking, deduplication, authority scoring, and query routing.
Context fidelity	Retrieved text inserted directly into prompt.	Some citation and context-selection logic.	Citation-aware context assembly with evidence support checks.
Orchestration governance	Ad hoc prompts and model calls.	Prompt versions and model routes tracked manually.	Versioned prompts, model routing, validation, fallback paths, tool policies, and approval rules.
Evaluation maturity	Demo examples only.	Offline set for common questions.	Layered evaluation for retrieval, grounding, citation fidelity, policy compliance, cost, latency, and drift.
Operational traceability	Basic application logs.	Partial model and application traces.	End-to-end traces from source ingestion to retrieval, context, model output, citation, and action.
Agentic action control	No tool use or unmanaged tool calls.	Tools scoped for selected workflows.	User-, task-, policy-, and approval-aware tool execution with audit trails.

Table 6. Readiness-score interpretation.

Average score	Interpretation
0.0-1.9	Prototype or demonstration system
2.0-2.9	Pilot-ready system
3.0-3.9	Governed but incomplete production system
4.0-4.5	Production-grade governed knowledge platform
4.6-5.0	Controlled agentic platform foundation

7.1 Worked readiness example

A worked example helps make the score less abstract. Suppose an internal policy assistant has moved beyond a curated pilot and now answers questions across HR policy, regional operating guidance, and service desk articles. It has some hybrid search and basic citations, but metadata is uneven and permission updates lag content updates. A reasonable assessment might look as follows.

Table 7. Example readiness scoring for an internal policy assistant.

Dimension	Score	Rationale
Ingestion coverage	3	Three repositories are connected, but service tickets and regional exceptions are still imported manually.
Metadata integrity	2	Document source and date are stored, but chunk-level effective dates and authority labels are incomplete.
Authorization fidelity	3	Role filters are applied during retrieval, but group changes are not propagated on a separate fast path.
Freshness management	2	Daily indexing is configured for policy documents, but supersession status is not consistently captured.
Retrieval robustness	3	Hybrid search is available, but query routing and authority-aware re-ranking are not yet tuned.
Context fidelity	3	Answers include citations, but claim-level evidence checks are limited.

Orchestration governance	2	Prompts are versioned in a repository, but routing, fallback, and refusal behavior are still manual.
Evaluation maturity	2	A small offline test set exists, but edge cases for conflicts, permissions, and freshness are thin.
Operational traceability	2	Application and model logs exist, but end-to-end traces from source to output are incomplete.
Agentic action control	1	The assistant is read-only; draft-ticket tooling is planned but not governed.

The average score in this example is 2.3, which places the system in the pilot-ready range rather than the production-grade range. That result is useful because it prevents a misleading conclusion: the assistant may already be valuable, but the architecture is not yet mature enough for broad deployment or action-oriented workflows. The gaps are specific: metadata integrity, freshness, traceability, evaluation, and action control need investment before the assistant should be scaled.

8. Evaluation Protocol for Governed Knowledge Systems

Evaluation in GKS-5 should test not only whether an answer is useful, but whether the platform preserved knowledge integrity across ingestion, indexing, retrieval, orchestration, and application delivery; recent RAG evaluation work similarly emphasizes retrieval quality, grounding, and answer faithfulness as distinct concerns [7], [8]. Standard answer-quality checks are not enough. A correct answer produced from unauthorized context is not acceptable. A fluent answer with unsupported citations is not trustworthy. A fast answer from stale content may be worse than a slower answer that warns about freshness.

A GKS evaluation protocol should deliberately exercise enterprise failure modes rather than only common user questions. At minimum, it should include the following test classes.

Citation tests should verify whether generated claims are supported by retrieved evidence, consistent with RAG evaluation approaches that assess grounding and faithfulness [7], [8].

Each evaluation case should define expected evidence, allowed source classes, prohibited source classes, required metadata filters, expected answer behavior, and expected refusal or escalation conditions. The test should verify not only the final answer, but also the path by which the answer was produced.

For example, a freshness test should specify which source version is current, which older versions are superseded, how the system should rank them, and whether the final response should disclose evidence freshness. A permission test should verify that restricted content was not retrieved into model context, not merely that it was absent from the final answer. An authority test should verify that approved sources outrank informal notes even when the informal note is semantically closer to the user's question.

The protocol separates evaluation into three levels:

Layer-level evaluation: ingestion coverage, metadata quality, chunk quality, retrieval relevance, authorization filtering, and index freshness.

Response-level evaluation: grounding, citation support, answer correctness, refusal behavior, uncertainty handling, and source transparency.

Workflow-level evaluation: tool use, approval behavior, auditability, rollback, task completion, and policy compliance.

This separation matters because an end-to-end answer score can hide the root cause of failure. GKS evaluation therefore treats quality, governance, and operational behavior as inseparable. The goal is not only to measure whether the system answered correctly, but whether it answered from the right evidence, under the right permissions, with enough traceability to explain and improve the result.

Together, the taxonomy, readiness score, and evaluation protocol create a closed loop: diagnose failure modes, assess architectural maturity, and test whether the system preserves knowledge integrity under production-like conditions.

Table 8. Representative evaluation classes for governed knowledge systems.

Test class	What it evaluates	Expected behavior
------------	-------------------	-------------------

Freshness tests	Whether the system prefers current, non-superseded sources.	Retrieve current evidence; warn or refuse when only stale evidence is available.
Permission tests	Whether restricted content is excluded before model context assembly.	Do not retrieve, rank, or use content outside the user entitlement.
Authority tests	Whether official sources outrank informal, duplicate, or draft sources.	Prefer approved policies and canonical records over unofficial or duplicate content.
Conflict tests	Whether contradictory sources are handled safely.	Identify the conflict, scope the answer, prefer authoritative sources, or escalate.
Identifier tests	Whether exact terms, codes, clauses, ticket IDs, and dates are handled.	Use lexical or hybrid retrieval rather than semantic similarity alone.
Citation tests	Whether each material claim is supported by cited evidence.	Return citations that directly support the claim or mark the answer as unsupported.
Tool-use tests	Whether action execution obeys user, task, policy, and risk constraints.	Use scoped tool permissions, approval gates, action logs, and rollback paths.
Cost and latency tests	Whether quality is obtained within acceptable operational boundaries.	Track retrieval depth, reranking cost, context length, model routing, and response time.

9. Scenario-Based Validation: Policy Assistant Expansion

This section illustrates how GKS-5 can be used as an assessment and design framework. The scenario is a common enterprise pattern: an internal policy assistant expands from a narrow pilot into a multi-domain production system.

The scenario is representative rather than organization-specific. A business unit first builds a retrieval-augmented assistant over a curated set of policy documents. The pilot performs well because the corpus is small, permissions are simple, documents are manually selected, and evaluation is informal. Encouraged by early adoption, the organization expands the assistant to include HR policies, compliance guidance, regional operating procedures, internal wiki pages, support knowledge, legal summaries, and workflow actions such as drafting requests or opening tickets.

This expansion exposes conditions that were hidden during the pilot: duplicate documents, stale policies, conflicting regional guidance, inconsistent permissions, weak metadata, and user questions that combine policy interpretation with operational context. The scenario is useful because it tests the failure modes that often separate a convincing prototype from a production-grade enterprise AI system.

The scenario is assessed across three architectural approaches: Basic RAG, Governed RAG, and GKS-5. Basic RAG can work well over a curated pilot corpus, but it tends to degrade as sources, permissions, and document types expand. Governed RAG improves reliability by adding metadata filters, citations, access-control checks, and some evaluation around the retrieval pipeline. GKS-5 goes further by treating ingestion, indexing, retrieval, orchestration, application integration, and cross-layer controls as platform responsibilities rather than pipeline enhancements.

The evaluation uses eight test classes from the GKS evaluation protocol: freshness, permission, authority, conflict, identifier, citation, agentic action, and regression. Each class represents a failure mode commonly observed when enterprise AI systems move from controlled pilots to broader production use. The comparison is conceptual rather than empirical. Its purpose is not to claim measured performance from a deployed system, but to show how the framework can be used to reason about architectural behavior and identify capabilities required for production readiness. Detailed scenario evaluation results are provided in Appendix A.

The same scenario can also be assessed using the Governed Knowledge System Readiness Score introduced in Section 7. In an illustrative application of the score, Basic RAG falls in the prototype or demonstration range, Governed RAG falls in the governed but incomplete production range, and GKS-5 falls in the controlled agentic platform foundation range. These scores are illustrative rather than measured performance results. They show how the readiness model can be applied to compare architectural maturity under the same expansion scenario. Detailed scoring is provided in Appendix B.

The most important distinction is diagnostic. In Basic RAG, a stale or unauthorized answer is often treated as a prompt, model, or retrieval-tuning problem. In Governed RAG, the same failure may be diagnosed within the retrieval pipeline. In GKS-5, the failure can be traced across the relevant layer and control plane: stale answers point to the freshness control plane, unauthorized grounding points to authorization, authority inversion points to metadata and retrieval, and citation mismatch points to context assembly and evaluation.

This diagnostic capability is central to the paper's argument. Production enterprise AI systems do not only need better answer generation. They need a way to locate, explain, and correct failures across the knowledge path.

The scenario does not prove that GKS-5 is the only possible architecture for enterprise AI knowledge systems. It demonstrates the kind of failure coverage that a production architecture must provide. A narrow RAG pipeline can be sufficient for a controlled pilot. A governed RAG pipeline can support a bounded production use case. A broader Governed Knowledge System becomes necessary when the system must scale across heterogeneous sources, changing permissions, source-authority conflicts, freshness requirements, evaluation obligations, and controlled agentic workflows.

The value of GKS-5 is not that it replaces RAG, LLMOps, data fabric, or agentic frameworks. Its value is that it organizes these capabilities into a platform model that can preserve knowledge integrity as enterprise AI systems expand from assistants to operational systems.

Scope of the Scenario Validation

The scenario-based validation in this section is illustrative rather than empirical. It is intended to show how GKS-5, the readiness score, and the evaluation protocol can be applied to a representative enterprise AI expansion scenario. The comparison does not report measured performance from a deployed system, and it does not claim that GKS-5 will outperform other architectures in every environment.

Future work should apply the readiness score to deployed enterprise AI systems, compare observed production failures against the failure-mode taxonomy, and evaluate whether GKS-5 improves reliability, governance, traceability, and operational outcomes in practice.

Table 9. Scenario-based validation of GKS-5 against common expansion failures.

Test class	Example test condition	Expected GKS-5 behavior versus basic RAG
Freshness	User asks about a policy superseded by a newer version.	Basic RAG may retrieve stale policy if semantically similar. GKS-5 uses freshness metadata, supersession status, and freshness-aware retrieval.
Permission	User asks about a restricted regional policy.	Basic RAG may retrieve restricted content from a shared index. GKS-5 enforces authorization before retrieval, context assembly, and tool use.
Authority	Official policy conflicts with an informal wiki explanation.	Basic RAG may rank the wiki higher if semantically closer. GKS-5 uses source-authority scoring and canonical source tags.
Conflict	Approved policies give different guidance for different regions.	Basic RAG may blend guidance. GKS-5 scopes by jurisdiction metadata and escalates when conflict remains.
Identifier	User asks about a clause, product code, control ID, or ticket number.	Basic RAG may miss exact matches. GKS-5 combines lexical, semantic, and metadata-aware routing.
Citation	Generated answer includes several procedural claims.	Basic RAG may cite the nearest document. GKS-5 connects claims to supporting passages and tests citation fidelity.
Action	Assistant drafts or opens an operational ticket.	Basic agentic extensions may overreach. GKS-5 requires tool policies, approvals, audit trails, and rollback paths.

10. Implementation Mapping: Managed Cloud and Open-Source Patterns

The proposed architecture is intentionally vendor-neutral. GKS-5 can be implemented with managed cloud services, open-source components, self-hosted infrastructure, or hybrid deployment models. Relevant implementation building blocks include observability frameworks, policy engines, and data-security platforms such as OpenTelemetry, Open Policy Agent, and Apache Ranger [22], [23], [24]. The purpose of this section is not to recommend a fixed stack. It is to show how architectural responsibilities can be mapped to implementation capabilities without allowing product choices to define the architecture.

A useful implementation strategy begins with responsibilities rather than tools. A Governed Knowledge System needs ingestion, durable storage, semantic indexing, lexical search, orchestration, compute, governance, observability, evaluation, and application integration. Different organizations will implement these capabilities differently

depending on regulatory requirements, team maturity, deployment constraints, cost posture, data residency requirements, and existing enterprise platforms.

A representative mapping of GKS-5 responsibilities to managed-cloud and open-source implementation patterns is provided in Appendix C. The mapping is illustrative rather than prescriptive; its purpose is to show how architectural responsibilities can be implemented without allowing product choices to define the architecture.

This mapping reinforces a key design principle: enterprise AI platforms should be designed around responsibilities, not products. Managed services can be a rational choice when speed, operational simplicity, enterprise support, or compliance alignment matters. Open-source components can be equally valuable where transparency, portability, customization, or deployment control is more important. Most production environments will use a mixture of both.

The architectural risk is not the use of managed services. The risk is allowing a service, framework, or model endpoint to erase the boundaries between layers. An enterprise may choose managed object storage and managed model endpoints while keeping retrieval logic, metadata contracts, and orchestration interfaces portable. Another organization may prefer open-source indexing and orchestration components while relying on managed identity, compute, or monitoring services. Both approaches can be sound if the platform preserves clear interfaces and avoids unnecessary coupling.

The right balance depends on context. Smaller teams may benefit from managed services because they reduce infrastructure burden and accelerate delivery. Regulated or sovereign environments may need greater control over deployment, data residency, model hosting, and policy enforcement. Large enterprises may need hybrid patterns that allow some workloads to run in managed cloud environments while more sensitive workloads run in controlled or on-premises infrastructure.

The important question is not whether managed or open-source tools are inherently better. The important question is whether the architecture preserves the ability to evolve. Requirements will change. Models will change. Retrieval methods will change. Governance expectations will change. A platform designed around stable responsibilities and explicit boundaries can absorb those changes. A platform designed around the first successful prototype usually cannot.

Table 10. Representative implementation responsibilities for GKS-5.

Architecture responsibility	Generic capability	Key design concern
Ingestion	Connectors, ETL/ELT, streaming, change capture.	Freshness, provenance, source coverage, metadata capture.
Raw and processed storage	Object storage, lakehouse, content repository.	Durability, lineage, reprocessing, raw/processed separation.
Semantic indexing	Embedding storage and vector retrieval.	Embedding lifecycle, partitioning, recall, latency, index updates.
Lexical search	Full-text indexing and keyword retrieval.	Exact matching, explainability, hybrid retrieval, operational identifiers.
Retrieval service	Query routing, metadata filters, re-ranking, source authority.	Authorization boundary, ranking transparency, context assembly.
Model orchestration	Model gateway, routing, tool control, workflow coordination.	Prompt/version management, validation, fallback behavior, tool invocation.
Policy enforcement	Entitlements, ABAC/RBAC policies, approval gates.	Pre-retrieval enforcement, tool permission scoping, auditability.
Observability	Tracing, metrics, logging, evaluation dashboards.	End-to-end diagnostic visibility, regression detection, cost monitoring.

11. Design Trade-offs

GKS-5 separates responsibilities, but it does not remove trade-offs. Enterprise AI platforms operate under competing constraints: users want fast answers, security teams want strict control, business units want broad coverage, infrastructure teams want manageable cost, and architects need flexibility as models, retrieval methods, and deployment requirements change.

A useful platform does not pretend these tensions disappear. It gives teams explicit places in the architecture where trade-offs can be managed: retrieval depth, freshness policy, model routing, metadata contracts, orchestration controls, deployment boundaries, and domain-level specialization.

11.1 Latency Versus Answer Quality

One of the most persistent trade-offs in enterprise AI is the tension between responsiveness and grounding quality. Better answers often require more work before generation: broader retrieval, authorization checks, metadata filtering, re-ranking, deduplication, source-authority resolution, context assembly, and sometimes a more capable model. Each step can improve trust, but each also adds latency and cost.

This trade-off is especially visible in interactive systems. Users expect conversational responsiveness even when the platform is searching across multiple indexes, enforcing entitlements, resolving source authority, and synthesizing evidence from different repositories. Shallow retrieval may improve speed, but it increases the risk of incomplete, stale, or misleading answers. Deeper retrieval may improve coverage, but it can slow the experience and increase context size.

There is no universal latency target. A service desk assistant used during a live interaction may need speed. A compliance research assistant can reasonably take longer if the answer is better grounded and cited. An agentic workflow that initiates an operational action may require additional checks even if they delay completion.

The platform response should be adaptive. Retrieval depth, re-ranking strategy, model choice, caching, and validation should vary by task type, risk level, user context, and expected use. The goal is not always to minimize latency. The goal is to spend latency where it improves trust, safety, or task completion.

11.2 Freshness Versus Cost

Freshness is central to trust. Users quickly lose confidence in systems that answer from superseded policies, stale tickets, outdated procedures, or old operational records. But freshness is not free. Frequent synchronization, parsing, embedding, indexing, permission propagation, and cache invalidation increase cost and operational complexity.

The mistake is treating freshness as a platform-wide setting. Enterprise sources change at different rates and carry different risks when stale. A quarterly policy manual does not need the same ingestion pattern as an active incident record. A pricing table may require faster updates than a training guide. Permission changes may need to propagate faster than content changes because stale entitlements create security risk even when the document itself has not changed.

A mature platform defines freshness classes and assigns ingestion strategies accordingly. Some sources can use scheduled batch updates. Others need incremental updates, event-driven ingestion, or near-real-time indexing. Some content can tolerate delayed embedding but requires immediate metadata or access-control updates. This distinction reduces cost while making freshness behavior intentional.

Freshness should also be visible downstream. Retrieval and orchestration should know when content was last indexed, whether a source is authoritative, and whether the available evidence is fresh enough for the user's question. In some cases, the correct behavior is not to answer confidently, but to warn that the indexed knowledge may be stale.

11.3 Managed-Service Productivity Versus Long-Term Portability

Managed services often provide the fastest path to a working system. They reduce infrastructure burden, provide operational support, and allow teams to focus on use cases instead of low-level platform engineering. For small teams or early pilots, that speed can be decisive.

The trade-off appears later. Systems that depend too deeply on proprietary orchestration abstractions, model-serving contracts, indexing interfaces, or service-specific metadata formats can become difficult to migrate, extend, or govern consistently. What helped a team move quickly during the pilot may later constrain cost management, deployment flexibility, cross-cloud strategy, or compliance posture.

Portability should not be treated as an ideological requirement that every component must be interchangeable at all times. That is rarely realistic. The practical goal is to preserve credible migration paths. An organization may choose managed model endpoints, search services, or storage while keeping retrieval logic, metadata contracts, evaluation data, and orchestration policies explicit enough to move or reimplement later.

The lesson is not to avoid managed services. It is to use them behind clean interfaces and stable contracts. Managed services should accelerate the platform, not define its architecture.

11.4 Generality Versus Specialization

A shared enterprise AI platform can reduce duplication and create reusable capability across teams. Common ingestion patterns, retrieval services, orchestration controls, governance mechanisms, and observability pipelines help organizations avoid building a separate stack for every assistant, copilot, or agent.

Excessive uniformity, however, reduces quality. Different domains behave differently. Legal text, engineering documentation, policy manuals, support tickets, scientific reports, medical records, financial controls, and software logs may require different chunking strategies, metadata models, ranking rules, citation expectations, and prompt framing. A platform that forces every domain through the same retrieval and orchestration path will eventually underperform.

The opposite failure is uncontrolled specialization. If every team builds its own connectors, indexes, prompts, evaluation methods, and access-control logic, the enterprise ends up with fragmented systems that are difficult to govern and expensive to maintain.

The better pattern is selective specialization within shared boundaries. The platform should provide common identity integration, metadata standards, governance controls, observability, evaluation, orchestration policies, and reusable retrieval services. Domains should specialize where it materially improves quality: chunking, ranking, metadata enrichment, source-authority rules, terminology handling, and user experience. This preserves reuse without flattening domain differences.

11.5 Sovereignty and Deployment Flexibility

Enterprise AI platforms in regulated, public-sector, defense, healthcare, financial, and critical infrastructure environments often face constraints that differ from commercial deployments. These may include data residency requirements, controlled infrastructure, restricted network connectivity, audit obligations, local model hosting, and air-gapped operation.

In these contexts, platform design must assume deployment mobility. A system may need to run in public cloud for some workloads, sovereign cloud for others, and on-premises for the most sensitive use cases. It may also need to substitute managed model APIs with private endpoints or locally hosted models. If the architecture is tightly bound to one environment, meeting these requirements can require a major redesign.

A layered architecture helps because responsibilities can move independently. Ingestion can adapt to local data sources and connectivity constraints. Storage and indexing can run in sovereign or isolated environments. Retrieval logic can remain consistent even when the underlying search infrastructure changes. Orchestration can abstract model providers so that managed APIs, private endpoints, and local models operate under a common control model.

The most significant risk to portability is often not the infrastructure itself. It is the assumptions embedded in retrieval pipelines, metadata schemas, orchestration logic, and application code. If application interfaces directly depend on a specific vector store, model provider, metadata format, or cloud service, migration becomes expensive. If those assumptions are contained behind stable contracts, the platform has more room to adapt.

Portability in enterprise AI should therefore mean preserving credible migration and deployment paths, not pretending every component is perfectly interchangeable. This matters where regulatory expectations, procurement requirements, or national data policies may change over time. A compact summary of the trade-offs discussed in this section is provided in Appendix D.

Table 11. Summary of design trade-offs.

Trade-off	Benefit optimized	Constraint introduced	Platform response
Latency vs. answer quality	Faster responses or deeper grounding.	Balancing user experience, cost, completeness, and risk.	Adaptive retrieval depth, caching, re-ranking policies, and tiered model selection.
Freshness vs. cost	More current and trusted information.	Synchronization, embedding, indexing, permission propagation, and cache invalidation overhead.	Freshness classes by source, content type, update rate, and risk level.

Managed services vs. portability	Faster delivery and reduced operational burden.	Migration complexity, vendor lock-in, and long-term control.	Stable interfaces, metadata contracts, and modular layer boundaries.
Generality vs. specialization	Reuse across teams and shared governance.	Domain-specific terminology, ranking, citation, and user-experience needs.	Common platform services with controlled domain-level specialization.
Sovereignty and deployment flexibility	Compliance, data residency, and operational control.	Deployment complexity and duplicated infrastructure.	Hybrid deployment, local model options, and portable retrieval/orchestration contracts.

12. Implementation Considerations

A reference architecture is only useful if it survives implementation. In enterprise AI, the hardest problems often appear after the first working system is already in use: sources change, permissions shift, indexes grow, users ask less predictable questions, and the cost of operating the platform becomes visible. This section summarizes implementation concerns that should be treated as design requirements, not late-stage refinements.

12.1 Data Freshness and Re-indexing

Users lose trust quickly when an AI system answers confidently from stale or superseded content. Freshness should therefore be treated as an explicit platform property. Each source should have a freshness expectation based on its rate of change, business importance, governance risk, and user-facing use cases.

Not every source needs the same update pattern. A policy repository updated quarterly may be served by scheduled ingestion. Active incident records, customer cases, pricing data, or operational telemetry may require incremental updates or event-driven processing. Permission changes may need to propagate faster than content changes because stale entitlements create risk even when the underlying document has not changed.

Re-indexing should also be selective. A content change, metadata correction, and entitlement update have different downstream implications. A content change may require parsing, chunking, embedding, and index refresh. A metadata update may only affect filters or ranking signals. A permission update may require immediate access-control projection without re-embedding the content. Treating every update as full reprocessing wastes compute, increases operational disruption, and slows the platform as the corpus grows.

Freshness must also be visible at retrieval and orchestration time. The system should know when evidence was last indexed, whether a source is authoritative, and whether available context is current enough for the question being asked. In some cases, the right behavior is to answer with a freshness warning. In others, it is to refuse a confident answer until the relevant source is refreshed.

12.2 Cost of Retrieval Failure

Retrieval failure is one of the easiest costs to underestimate in enterprise AI. A poor retrieval result does not remain isolated. It propagates through infrastructure usage, model cost, human review effort, and user trust.

At the infrastructure layer, weak retrieval creates compute amplification. Systems compensate by retrieving larger candidate sets, performing additional re-ranking, issuing follow-up searches, or expanding context windows. These steps increase latency and resource usage, often without solving the underlying quality problem.

At the model layer, missing or poorly ranked evidence leads to inefficient generation. The platform may call a larger model, pass more tokens, retry the request, or ask the model to reason over irrelevant context. This increases cost while giving the appearance of deeper reasoning. In reality, the system may be spending more to work around a retrieval problem.

The largest cost often appears at the human layer. Users verify answers, search manually, correct outputs, escalate uncertain cases, or stop using the system. In operational settings, these correction costs can exceed infrastructure costs because they consume expert time and slow business workflows. A system that is cheap to run but expensive to trust is not actually cheap.

Retrieval quality, metadata quality, and source authority are therefore economic concerns, not only technical ones. Investments in ingestion, chunking, metadata propagation, ranking, evaluation, and observability often produce more value than incremental improvements in model capability. A stronger model can improve expression and reasoning, but it cannot reliably recover evidence the platform failed to retrieve.

12.3 Observability and Quality Monitoring

Enterprise AI platforms require observability across the full path from source ingestion to final response; vendor-neutral telemetry standards provide one implementation pattern for distributed tracing, metrics, and logs [22]. Basic system health is not enough. Teams need to understand how the system behaved, why it behaved that way, and what changed when quality, latency, cost, or compliance posture shifted.

Useful signals include ingestion lag, parse failures, indexing backlog, embedding failures, retrieval latency, candidate-set size, filter behavior, ranking quality, context size, model latency, token cost, answer acceptance, citation usage, escalation frequency, policy events, and user feedback. These signals are useful individually, but their real value comes from correlation across layers.

A mature platform should be able to trace a response through the system: which sources were available, when they were last indexed, how they were parsed, which query interpretation was used, which filters were applied, which retrieval path was selected, which passages were ranked highly, what context was passed to the model, which model or tool path was used, and what validation or policy checks were applied.

This diagnostic clarity matters because enterprise AI failures are often misclassified. A user may report a hallucination when the real issue was stale content, a parsing error, a missing entitlement, a poor chunk boundary, a weak ranking threshold, or a prompt change that altered how evidence was used. Observability makes these causes inspectable rather than anecdotal.

Monitoring must also account for drift. Retrieval quality can decline as corpora grow. Citation fidelity can weaken when metadata becomes inconsistent. Prompt costs can rise as context windows expand. Latency can increase as orchestration logic becomes more complex. User trust can decline before infrastructure metrics show a clear outage. Quality monitoring should therefore combine operational metrics, evaluation signals, and user-facing outcomes.

Observability also has a governance role. Enterprises often need evidence of which sources influenced an answer, whether restricted content was filtered correctly, which tool calls were attempted, whether a user had the necessary entitlement, and whether policy checks occurred before action was taken. A mature observability model therefore combines operational monitoring, diagnostic tracing, quality evaluation, cost visibility, and governance evidence.

12.4 Security and Access Control

Security in enterprise AI cannot be confined to the user interface or treated as a final filtering step. Security controls and zero-trust principles support the argument that access decisions should be enforced close to the resource and evaluated continuously across the request path [20], [21]. Authorization, sensitivity labels, tenant boundaries, and policy constraints must propagate through ingestion, indexing, retrieval, orchestration, and application delivery. If access control is applied only after retrieval, the system may already have used restricted content to shape an answer.

Role-based access control is often necessary, but it is rarely sufficient. Enterprise environments commonly require document-level entitlements, attribute-based controls, tenant isolation, geography-specific restrictions, source-specific rules, and sensitivity classifications. These constraints should influence which content is indexed, which candidates are retrieved, which passages are assembled into context, and which actions the system is allowed to take; policy engines and data-security frameworks provide implementation patterns for this kind of enforcement [23], [24].

This is especially important in retrieval-augmented systems. A user may not see a restricted passage in the final answer, but if that passage influenced generation, the security boundary has already been weakened. Retrieval must therefore be authorization-aware before context reaches the model. In sensitive environments, the platform should also preserve evidence of which policies were evaluated, which filters were applied, and which sources were excluded.

Agentic systems expand the security problem further. Read permissions are not enough when a system can invoke tools, call APIs, draft records, open tickets, approve requests, or trigger downstream workflows. Tool access should be scoped by user identity, task type, data sensitivity, approval requirements, and execution context. Some actions may be allowed automatically; others may require human confirmation or be prohibited entirely.

This shifts security from static access control to dynamic execution control. The platform must evaluate permissions as the workflow progresses, especially when the system retrieves new information, changes plans, or moves from

recommendation to action. For high-consequence workflows, audit trails should record the user context, retrieved evidence, tool calls, policy checks, approvals, and final outcome.

A secure enterprise AI platform treats access control as part of the architecture, not as a wrapper around the application. Security must shape retrieval, constrain orchestration, and govern action.

12.5 Prompt and Workflow Versioning

Prompt design, retrieval configuration, routing logic, model selection, tool permissions, validation rules, and fallback behavior should be treated as versioned operational assets. They are not informal settings. Small changes can affect answer quality, cost, latency, policy behavior, and user trust.

Versioning matters because AI systems rarely fail in only one place. A prompt change may improve fluency while weakening citation discipline. A new retrieval threshold may improve recall while increasing irrelevant context. A model swap may reduce latency but change refusal behavior. A tool-permission update may make a workflow more useful while increasing execution risk. Without version control, testing, and rollback, teams cannot reliably understand what changed or recover when quality declines.

This is particularly important for agentic workflows. When a system can plan steps, call tools, and interact with enterprise APIs, workflow logic becomes part of the control plane. Changes to routing, approval rules, tool scopes, retry behavior, or escalation paths can produce downstream effects that are difficult to predict from a single test case. These configurations should be reviewed, tested, deployed, and monitored with the same discipline applied to production software.

Versioning also enables controlled experimentation. Teams can compare prompt variants, retrieval strategies, model choices, and workflow policies against the same evaluation set. They can measure not only whether an answer sounds better, but whether grounding, citation accuracy, latency, cost, refusal behavior, and policy compliance improve or degrade. This turns prompt and workflow iteration from informal tuning into managed engineering practice.

A mature platform should preserve version history for prompts, retrieval parameters, model routes, tool policies, validation rules, and evaluation results. This history becomes essential for debugging, auditability, and safe change management.

12.6 Evaluation as Release Discipline

A production enterprise AI platform requires evaluation beyond successful demonstrations. RAG evaluation frameworks and benchmarks increasingly separate retrieval relevance, grounding, answer quality, and failure analysis rather than treating the system as a single black box [7], [8]. A demo can show that the system works for selected examples. Evaluation must show whether it remains reliable across changing sources, diverse users, ambiguous questions, policy boundaries, and production workflows.

Evaluation should measure retrieval relevance, grounding, citation fidelity, policy compliance, latency, cost, task success, and user trust. No single metric captures all of these. Treating the system as one black box often hides the root cause of failure because an unreliable answer may originate in ingestion, metadata, retrieval, ranking, orchestration, generation, or application integration.

A key principle is to evaluate layers separately as well as end to end. Retrieval quality should be measured independently from generation quality. Many apparent hallucinations begin with missing, stale, unauthorized, or poorly ranked context. At the same time, strong retrieval does not guarantee a good answer if orchestration uses the evidence poorly or the model fails to cite it correctly. Mature platforms therefore evaluate ingestion coverage, metadata quality, chunk quality, retrieval precision and recall, ranking effectiveness, answer grounding, citation support, and task-level usefulness as related but distinct concerns.

Evaluation should combine offline and online methods. Offline evaluation provides repeatability. It allows teams to compare retrieval pipelines, prompt versions, embedding models, re-rankers, policy rules, and orchestration strategies against a stable set of questions and expected evidence. Online evaluation captures what curated test sets miss: ambiguous user behavior, workflow friction, reformulated questions, edge cases, and domain-specific expectations. In enterprise settings, online signals may include answer acceptance, citation use, reformulation rate, escalation frequency, task completion, and human correction effort.

Human review remains necessary, especially for high-consequence domains. Automated metrics can detect regressions and provide useful proxies for relevance or grounding, but they rarely capture whether an answer is operationally useful, appropriately scoped, or aligned with domain expectations. Sampled expert review is therefore part of responsible deployment, not an optional refinement.

Evaluation must be integrated into change management. Enterprise AI systems evolve continuously as sources, permissions, retrieval strategies, models, prompts, and policies change. Before deployment, teams should be able to compare what improved, what degraded, and whether the trade-off is acceptable. After deployment, monitoring should detect drift in retrieval quality, citation fidelity, latency, cost, policy behavior, and user trust.

Two mistakes are common. The first is delay: evaluation is added only after broad rollout, when weaknesses are more expensive to fix. The second is over-reliance on one aggregate score. A single score can be useful for reporting, but it rarely explains what to improve. Enterprise AI evaluation should balance technical quality, governance, reliability, cost, and business usefulness.

13. Discussion: Why Pilots Stall and What Platform Leaders Should Learn

Enterprise AI pilots often succeed under conditions that do not survive production. A narrow corpus, simple permissions, curated documents, manual oversight, and selected demo questions can make a system appear more mature than it is. The problem appears when the same assistant is expanded across broader sources, changing permissions, inconsistent metadata, duplicated content, and operational workflows.

Consider an insurer that launches an internal policy assistant in less than eight weeks. The pilot uses a carefully selected set of underwriting manuals, claims guidelines, and internal policy summaries. Within that narrow scope, the assistant answers common questions well enough to create confidence, executive interest, and pressure to expand.

The problems begin when the assistant moves beyond the original pilot boundary. New repositories are added from claims operations, compliance, customer service, legal, and regional business units. Each source has its own document standards, update cycles, ownership model, terminology, and access rules. Retrieval quality begins to decline. Some answers are grounded in superseded guidance. Others reflect duplicate or conflicting documents that were never reconciled during ingestion. In some cases, the system retrieves material that should have been restricted because business-unit permissions were not consistently captured upstream.

The first reaction is often to question the model. Teams try a stronger model, revise prompts, or expand the context window. These changes may improve surface behavior, but they do not address the underlying problem. The model is exposing weaknesses that already existed in the enterprise knowledge environment: poor metadata, unclear source authority, stale content, inconsistent permissions, and fragile retrieval logic. The pilot succeeded because the corpus was narrow, curated, and manually supervised. The rollout struggled because the organization had not built the governed knowledge platform required for broader use.

This pattern is common. Many enterprise AI efforts succeed as demonstrations and stall as platforms. The lessons are consistent.

13.1 Retrieval quality shapes perceived intelligence

In early pilots, model selection often receives the most attention. In production, user trust depends more heavily on whether the system retrieves the right evidence. A strong model grounded in stale, incomplete, unauthorized, or poorly ranked context will still produce unreliable answers. Conversely, disciplined retrieval can make even a modest model more useful.

13.2 Metadata discipline is a platform requirement

Metadata is not an administrative detail. It determines whether the system can identify source authority, apply access controls, distinguish current documents from superseded ones, filter by domain or jurisdiction, and explain where an answer came from. Ownership, document type, effective date, confidentiality, lineage, and access attributes should be captured at ingestion time and preserved through indexing, retrieval, orchestration, and citation.

13.3 Enterprise retrieval must be hybrid

Pure semantic retrieval is useful for conceptual questions, but enterprise users often ask operationally precise questions involving identifiers, policy clauses, product codes, legal terms, ticket numbers, control references, or effective dates. These questions require semantic search, lexical matching, metadata filters, ranking, and source authority to work together. A single retrieval method rarely performs well across the full range of enterprise use cases.

13.4 Authorization must shape retrieval

Access control should be enforced before content is assembled into model context. Filtering or redacting after retrieval is not sufficient because restricted content may already have influenced generation. Production systems need authorization-aware candidate selection, document-level entitlements, tenant isolation, and source-specific restrictions built into the retrieval path itself.

13.5 Observability determines whether the system can improve

Enterprise AI systems fail in layered ways. A poor answer may originate from a parsing error, stale index, missing metadata field, weak ranking rule, prompt change, model swap, or access-control mismatch. Without end-to-end tracing from ingestion through retrieval, orchestration, and response generation, teams can see that quality declined but cannot reliably explain why. Systems without observability tend to degrade quietly until users lose trust.

13.6 Orchestration becomes a control layer as systems become agentic

In a simple assistant, orchestration may appear to be prompt assembly. In an agentic system, it governs model routing, tool invocation, permission checks, approval flows, fallback behavior, and auditability. Once AI systems can call APIs, draft records, open tickets, or initiate workflow actions, orchestration is no longer a convenience layer. It becomes part of the enterprise control surface.

13.7 Evaluation must begin before scale

13.8 What platform leaders should watch first

The first warning sign is usually not a bad answer. It is a team that cannot explain why the answer was produced. If the team cannot show the source eligibility decision, the retrieval candidates, the ranking signals, the context package, the prompt version, and the citation evidence, then quality improvement becomes guesswork. Model substitution may help, but it will not diagnose stale metadata, weak chunking, poor source authority, or permission drift.

The second warning sign is duplication of the knowledge stack. If every assistant has its own connectors, embedding job, vector index, prompt code, feedback logs, and permission logic, the enterprise does not have an AI platform. It has a portfolio of fragile applications. GKS-5 is intended to make those common responsibilities visible enough to be reused, governed, measured, and improved.

14. Threats to Validity, Limitations, and Future Work

This paper proposes GKS-5 as a reference architecture for Governed Knowledge Systems. It synthesizes recurring enterprise AI failure patterns and organizes them into a five-layer model with cross-layer controls, a failure-mode taxonomy, a readiness score, and an evaluation protocol. The work is intended as a design framework, not a universal implementation prescription.

The paper has several limitations. It does not present a quantitative benchmark, controlled user study, or empirical comparison of specific product stacks. The scenario-based validation is illustrative rather than measured. The readiness scores are intended to show how the framework can be applied, not to prove performance superiority. The proposed layers and controls may also need adaptation in environments with unusually constrained infrastructure, highly specialized knowledge domains, or strict regulatory requirements.

Future work should validate the proposed architecture through longitudinal case studies, production evaluations, and comparative analysis across regulated and non-regulated environments. Several directions are especially important.

First, the readiness score should be applied to deployed enterprise AI systems across different industries. This would help test whether the dimensions predict production reliability, governance maturity, and operational outcomes.

Second, the failure-mode taxonomy should be compared against observed incidents from real deployments. This would help refine the mapping between user-visible symptoms, platform root causes, and control-plane failures.

Third, evaluation protocols for governed retrieval should be developed further, extending current RAG evaluation and benchmarking work toward enterprise-specific concerns such as permissions, freshness, source authority, and action safety [7], [8]. Current AI evaluation often emphasizes answer quality or model performance. Enterprise AI evaluation also needs tests for freshness, permission fidelity, source authority, citation support, policy behavior, and action safety.

Fourth, adaptive retrieval remains an important research and engineering direction. Mature systems will need to vary retrieval depth, ranking strategy, context assembly, and model routing based on query type, user role, risk level, source authority, and workflow context.

Fifth, multimodal enterprise knowledge requires more systematic treatment. Important enterprise context often lives in tables, diagrams, scanned forms, engineering drawings, dashboards, spreadsheets, images, audio, video, meeting transcripts, and structured records. Future Governed Knowledge Systems will need stronger methods for ingesting, indexing, retrieving, citing, and evaluating these forms together.

Sixth, agentic workflows require stronger execution-control frameworks as reasoning, tool use, external knowledge access, and multi-step action become more tightly coupled [15], [16], [17], [18]. As AI systems move from answering questions to taking action, enterprises will need more precise models for tool permissions, approval flows, delegation, rollback, state management, audit trails, and human oversight.

Seventh, governance should become more executable. Access rules, retention requirements, jurisdictional restrictions, approval paths, source authority rules, and action boundaries should be encoded into platform behavior, not maintained only as external policy documents. The relevant question is not only what policy says, but how the system enforces it during ingestion, retrieval, orchestration, and execution.

The broader implication is that enterprise AI advantage will not come simply from adopting the newest model. Models will continue to improve, but durable advantage will come from converting model progress into governed, portable, reusable, and operationally trustworthy capability. That is an architectural problem. It is why platform design will remain central even as the underlying AI technology continues to change.

From a research-design perspective, the framework has construct, internal, and external validity limits. Construct validity depends on whether the proposed dimensions capture the operational properties that actually explain enterprise AI failures. Internal validity is limited because the paper does not isolate causal effects through controlled deployment experiments. External validity is limited because enterprise knowledge environments differ by domain, regulatory context, source maturity, and operating model. These limits do not invalidate GKS-5 as a reference architecture; they define the empirical work required to test and refine it.

15. Minimum Requirements for a Production Governed Knowledge System

A system should not be considered production-grade simply because it combines retrieval with a language model. A production Governed Knowledge System should, at minimum, provide the following capabilities:

- Source-level and chunk-level provenance so that retrieved evidence can be traced back to authoritative source material.
- Freshness metadata for retrievable content so that stale, superseded, or time-sensitive evidence can be identified and handled appropriately.
- Pre-retrieval authorization enforcement so that restricted content is excluded before it can influence model context or generation.
- Hybrid lexical-semantic retrieval so that the system can handle both conceptual questions and exact identifiers, clauses, codes, dates, and operational terms.
- Source-authority ranking so that approved, current, and canonical sources are preferred over informal, duplicate, draft, or obsolete content.
- Citation-aware context assembly so that material claims can be connected to the evidence used to support them.

- End-to-end traceability from response to source evidence so that teams can diagnose failures, audit behavior, and explain outputs.
- Versioning of prompts, indexes, retrieval parameters, models, policies, and orchestration flows so that behavior can be reproduced, compared, and rolled back.
- Offline regression evaluation and online quality monitoring so that retrieval quality, grounding, citation fidelity, policy compliance, latency, cost, and user trust can be tracked over time.
- Controlled tool use and approval gates for agentic workflows so that systems that act across enterprise tools do so under explicit user, task, policy, and risk constraints.

These requirements distinguish a production knowledge platform from a successful RAG demonstration and align with the broader need to combine retrieval, evaluation, security controls, and operational governance in enterprise AI systems [1], [7], [8], [19]-[21]. A prototype may omit some of them intentionally. A broad enterprise deployment should not omit them without explicit risk acceptance.

16. Conclusion

Enterprise AI is not simply a matter of connecting better models to enterprise data. The harder challenge is building systems that can ingest, authorize, retrieve, assemble, govern, evaluate, and operationalize enterprise knowledge reliably over time. This is why many enterprise AI efforts appear more mature in presentation than they prove to be in production. A polished interface can hide weak ingestion, incomplete metadata, inconsistent access control, stale content, and brittle retrieval during a pilot. It cannot hide them at scale.

This paper introduced GKS-5, a five-layer reference model for Governed Knowledge Systems. GKS-5 separates data ingestion, storage and semantic indexing, retrieval, model orchestration, and application integration. It pairs these layers with cross-layer controls for metadata, authorization, freshness, observability, evaluation, and versioning. The purpose of this structure is not to prescribe a single product stack, but to make the core responsibilities of enterprise AI knowledge systems visible, governable, and evolvable.

The practical value of this framing is diagnostic. In real deployments, failures that appear as hallucinations, stale answers, weak citations, unsafe recommendations, or poor task execution often originate upstream in the knowledge platform. They are frequently caused by weak source coverage, incomplete metadata, poor freshness management, shallow retrieval, inconsistent authorization, missing observability, or unversioned orchestration changes. Better models may reduce some symptoms, but they cannot replace the platform controls required to preserve knowledge integrity.

GKS-5 shifts attention from the assistant to the system beneath it. Enterprise AI assistants are rarely durable products by themselves. They are interaction surfaces over deeper knowledge platforms. Organizations that recognize this distinction may move more deliberately in the first phase, but they are better positioned to scale across broader sources, users, permissions, domains, and workflows.

The broader implication is strategic. Enterprises that treat AI as a thin conversational layer over disconnected content may produce fast demonstrations, but they will struggle to create durable capability. Enterprises that invest in platform-quality foundations can support not only search and assistance, but also richer reasoning, workflow integration, and governed agentic execution. They also retain the ability to adapt: to change models, refine retrieval methods, tighten governance, support new deployment environments, and add new business workflows without rebuilding the stack for every use case.

This is the difference between adopting AI as a feature and building AI as an enterprise capability. Features can be demonstrated quickly. Capabilities must survive scale, scrutiny, cost pressure, regulatory expectations, and organizational change.

Model capability will continue to improve, and the market will continue to reward speed. But speed without architecture usually produces fragile systems. The enterprises that benefit most from AI will not necessarily be the ones that deploy the largest number of assistants first. They will be the ones that build the strongest foundations for turning enterprise knowledge into governed, reusable, and operationally trustworthy intelligence.

Enterprise AI is therefore, at its core, a platform problem.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Online, Association for Computational Linguistics, 2020, pp. 6769-6781, doi: 10.18653/v1/2020.emnlp-main.550.
- [3] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009, doi: 10.1561/15000000019.
- [4] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39-48, doi: 10.1145/3397271.3401075.
- [5] T.-Y. Liu, "Learning to Rank for Information Retrieval," *Foundations and Trends in Information Retrieval*, vol. 3, no. 3, pp. 225-331, 2009, doi: 10.1561/15000000016.
- [6] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2021, doi: 10.1109/TBDATA.2019.2921572.
- [7] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, St. Julian's, Malta, Association for Computational Linguistics, 2024, pp. 150-158, doi: 10.18653/v1/2024.eacl-demo.16.
- [8] X. Yang, K. Sun, H. Xin, Y. Sun, N. Bhalla, X. Chen, S. Choudhary, R. D. Gui, Z. Jiang, Z. Jiang, L. Kong, B. Moran, J. Wang, Y. E. Xu, A. Yan, C. Yang, E. Yuan, H. Zha, N. Tang, L. Chen, N. Scheffer, Y. Liu, N. Shah, R. Wang, A. Kumar, W.-t. Yih, and X. L. Dong, "CRAG: Comprehensive RAG Benchmark," *arXiv:2406.04744*, 2024, doi: 10.48550/arXiv.2406.04744.
- [9] Z. Dehghani, "Data Mesh Principles and Logical Architecture," *MartinFowler.com*, Dec. 3, 2020. Accessed: May 11, 2026.
- [10] Z. Dehghani, *Data Mesh: Delivering Data-Driven Value at Scale*. Sebastopol, CA, USA: O'Reilly Media, 2022.
- [11] I. Blohm, F. Wortmann, C. Legner, and F. Köbler, "Data Products, Data Mesh, and Data Fabric: New Paradigm(s) for Data and Analytics?" *Business & Information Systems Engineering*, vol. 66, no. 5, pp. 643-652, 2024, doi: 10.1007/s12599-024-00876-5.
- [12] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [13] DAMA International, *DAMA-DMBOK: Data Management Body of Knowledge*, 2nd ed. Basking Ridge, NJ, USA: Technics Publications, 2017.
- [14] T. Berners-Lee, J. Hendler, and O. Lassila, "The Semantic Web," *Scientific American*, vol. 284, no. 5, pp. 34-43, May 2001.
- [15] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.
- [16] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [17] E. Karpas, O. Abend, Y. Belinkov, B. Lenz, O. Lieber, N. Ratner, Y. Shoham, H. Bata, Y. Levine, K. Leyton-Brown, D. Muhlga, N. Rozen, E. Schwartz, G. Shachaf, S. Shalev-Shwartz, A. Shashua, and M. Tenenholz, "MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning," *arXiv:2205.00445*, 2022.
- [18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [19] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.
- [20] National Institute of Standards and Technology, *Security and Privacy Controls for Information Systems and Organizations*, NIST Special Publication 800-53, Revision 5, Sep. 2020, doi: 10.6028/NIST.SP.800-53r5.
- [21] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, Aug. 2020, doi: 10.6028/NIST.SP.800-207.
- [22] OpenTelemetry Contributors, "OpenTelemetry Documentation," OpenTelemetry Project. Available: <https://opentelemetry.io/docs/>. Accessed: May 11, 2026.
- [23] Open Policy Agent Contributors, "Open Policy Agent Documentation," Open Policy Agent Project. Available: <https://openpolicyagent.org/docs/>. Accessed: May 11, 2026.
- [24] Apache Software Foundation, "Apache Ranger Documentation," Apache Ranger Project. Available: <https://ranger.apache.org/>. Accessed: May 11, 2026.